LayerTree: A Shared Bitcoin UTXO as an Ideal Functionality, and Its Realisation with Covenants

Olaoluwa Osuntokun

Lightning Labs
laolu@lightning.engineering

2026-10-09
Draft (commit d5d974a)
Full version

Abstract. Constructions that let many users share one Bitcoin output pay at exit time: a user who leaves publishes $O(\log N)$ transactions, the tree's interior must exist as pre-signed or template-committed transactions, and one unresponsive user forces the whole tree on-chain at expiry. We present LayerTree, in which a single output commits to an off-chain Merkle-sum tree of balances and tapscript covenants enforce every state transition. Any member exits, and the operator evicts any expired member, in *one* transaction whose size grows with the depth of the tree (about 32 virtual bytes per level) rather than with a path of transactions: 1,020 virtual bytes at depth 16. Membership changes need no interactive signing round, the shared output never expires, and whole cohorts are added or swept in one constant-shape transaction.

We state what the shared output does as an ideal functionality with interfaces JOIN, EXIT, EVICT, UPDATE and SWEEP, give an abstract protocol whose covenant predicates realise it, and compile the predicates to four opcode tiers built only from proposals already made for Bitcoin (restored string opcodes, introspection, stack signature checks, elliptic-curve tweaks, `OP_CHECKCONTRACTVERIFY`); the smallest tier needs `OP_CAT` and one point-tweak opcode, recovering introspection from a generator-key signature. We prove that the abstract protocol UC-realises the functionality in a global-ledger hybrid model, with the tree-level lemmas mechanised in Lean 4 and cited inline. Lightning channels compose with the functionality as leaves whose exit key is shared, so a hub of N channels occupies one output. A simulator, a byte-exact Go implementation and 885 executable checks accompany the design.

Keywords: Bitcoin · covenants · Merkle-sum trees · universal composability · payment channels · formal verification

1 Introduction

Payment channels [33] move Bitcoin activity off-chain for pairs; the open problem is doing it for *groups*: letting N users share one unspent transaction output

(UTXO) without giving up the ability of each to leave alone. Every deployed or proposed approach pays for the sharing when it is dissolved. In channel factories [5], CoinPool [34], Ark [20,21], timeout trees [22,45] and covenant trees built on `OP_CHECKTEMPLATEVERIFY` [35] or `OP_CHECKCONTRACTVERIFY` [18,17], the shared output is the root of a tree of *transactions*. A member who leaves unilaterally must publish the path from the root to its leaf, $O(\log N)$ transactions each paying a fee; the interior of the tree must exist in advance as pre-signed transactions (requiring every member to sign, and to be online) or as covenant templates; and when the tree expires, reclaiming the funds of a single unresponsive member means publishing everything.

This paper takes a different route. The tree stays off-chain and is a tree of *data*, not transactions: a binary Merkle-sum tree of balances whose root is committed by one output. A member exits by presenting its leaf and sibling path to the output itself; the spending script recomputes the root from the proof, checks it against the commitment, recomputes the root with the leaf removed, and requires the transaction to recreate the output under the new root with the leaf's amount paid out. The member needs nobody's signature, the exit is one transaction, and its size grows with the tree's depth rather than with a path of transactions. The same covenant lets the operator evict expired members, graft a pre-built subtree of new members in one step, and sweep an expired cohort cheaply.

We present the design from the interface inwards. Section 3 states, as an ideal functionality in the universal composability (UC) framework [6], what a shared output should let its members and operator do and what an adversary may influence. Section 4 gives an abstract protocol: a data structure, a set of covenant *predicates* over the spending transaction, and the behaviour of honest parties, independent of any particular opcode. Section 5 then realises the predicates against four opcode tiers: three descend from the full set of restored string opcodes (BIPs 440 and 441 [37,38]), introspection (`OP_TX` [36]), stack signature checks (`OP_CHECKSIGFROMSTACK` [3]) and elliptic-curve arithmetic [31,9] down to `OP_CAT` and one point-tweak opcode, and a fourth replaces the key arithmetic with `OP_CHECKCONTRACTVERIFY` [18]; several of these exist on Liquid [4]. We propose no opcode of our own. The separation lets the security argument be made once, for the predicates, rather than once per tier.

Contributions.

1. **The functionality** (§3). $\mathcal{F}_{\mathsf{LT}}$, a shared output with per-member balances, parameterised by depth, addressing profile, payout rule and a liveness bound $\Delta(N)$, with interfaces for unilateral exit, eviction, grafting, update, vouchers and sweeps.
2. **The abstract protocol** (§4). A Merkle-sum tree committed through a data-tweaked taproot key, and one *step* predicate from which every transition is composed, in an operator-assigned dense profile (the default) and a key-addressed sparse profile kept as the measured alternative.
3. **Realisation in four opcode tiers** (§5). Every predicate compiled against the full candidate set (FULL), every predicate but the cohort sweep against

a `0xC0`-only subset (MID), the single-leaf exit against `OP_CAT` and one point-tweak opcode by rebuilding the BIP-341 signature message in script and checking it with a generator-key signature (MIN), and every predicate but vouchers with `OP_CHECKCONTRACTVERIFY` in place of the key arithmetic (CCV). This is an upper bound on the opcodes needed; we prove no lower bound.

4. **A UC security proof with mechanised lemmas** (§6, §7). The abstract protocol UC-realises $\mathcal{F}_{\mathsf{LT}}$ in a hybrid model with a global ledger and clock and an ideal signature functionality, in the idealised hash model. The tree-level lemmas the simulator relies on are theorems in Lean 4, cited by their statements. The proofs isolated a fact the informal argument had blurred: sums are conserved unconditionally, but exits are guaranteed only under a real-tree invariant established off-chain.
5. **Composition with channels** (§9). Lightning channels hosted in leaves whose exit key is shared, with prevout-independent commitments emulating `ANYPREVOUT`, and the three conditions under which the channel functionality composes with $\mathcal{F}_{\mathsf{LT}}$.
6. **Implementation and measurement** (§11). A tapscript simulator, a byte-exact Go implementation, and measured costs: a 1,020 vB exit at depth 16, 665 vB per batched exit, 158 to 203 vB per swept leaf.

The specification, code and proofs are public [32]; this text describes revision d5d974a.

This version. This is the full version. It lifts into the body the UC proof (§7), the Lean development (§8) and the channel-leaf scripts (§10), and adds five sections: costs against bare UTXOs, Lightning and transaction trees (§12), the two profiles and the radix (§13), batch shapes (§14), the trust model of grafting (§15) and receiving over Lightning (§16). The short version keeps the first three as appendices and omits the five.

2 Background

Bitcoin. A pay-to-taproot output [42] commits to an internal key P and the Merkle root m of a *taptree* of scripts through $Q = P + H_{\mathsf{TapTweak}}(P \,\|\, m) \cdot G$, and is spent either by a Schnorr signature [41] under Q or by revealing one script with its Merkle path (the *control block*) and satisfying it. Tapscript [43] has no loops, so every "loop" below is unrolled and costs bytes per iteration. BIP-441 [38] defines tapleaf version `0xC2`, restoring the string opcodes disabled in 2010 (`OP_CAT`, `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT`, bit operations) with arithmetic on unsigned numbers of any length, metered by BIP-440 [37] ("varops"); BIP-347 [16] proposes `OP_CAT` alone in the current version. `OP_TX` [36] pushes selected fields of the spending transaction, optionally collated into one hash preimage; `OP_CHECKSIGFROMSTACK` [3] verifies a Schnorr signature over a stack element; `OP_TWEAKADD` [9] computes $x(P + tG)$, and the EC-ops draft [31] adds point addition, scalar multiplication and negation. All are proposals. A Merkle-sum tree [23,30] hashes each node's children with their sums, so an inclusion proof

proves a leaf's contribution to the root sum; sparse Merkle trees [10] address positions by hash.

Composable security. We use the UC framework [6] with global subroutines [8]. A protocol Π *UC-realises* a functionality $\mathcal{F}$ if for every adversary $\mathcal{A}$ there is a simulator $\mathcal{S}$ such that no environment $\mathcal{Z}$ distinguishes Π with $\mathcal{A}$ from $\mathcal{F}$ with $\mathcal{S}$. Following the composable treatment of Bitcoin by Badertscher et al. [2] we model the chain as a global ledger $\mathcal{G}_{\mathsf{ledger}}$ that accepts transactions satisfying a validity predicate, orders them at the adversary's discretion within a liveness bound Δ_L, and never reverts them (persistence); time is a global clock $\mathcal{G}_{\mathsf{clock}}$ [19] whose rounds are blocks; signatures are the ideal functionality $\mathcal{F}_{\mathsf{sig}}$ [7]. Channel functionalities in this model are given by [1,14], whose conventions §9 follows.

3 The LayerTree Functionality

A *tree* has one *operator* and any number of *members*. Each member owns a *leaf*: a balance and three public keys (an owner key that authorises the member's actions, an exit key paid on exit, and an evictor key, normally the operator's, that may remove the leaf after a lease height). The operator holds no key over the funds, only an *operator key* that authorises additions and co-signs cooperative changes. Figure 1 gives the functionality $\mathcal{F}_{\mathsf{LT}}$; the parameters are the depth D (the tree has 2^D slots), the addressing *profile* (dense, the default: the operator assigns slots; sparse, the measured alternative: a leaf's slot is a hash of its owner key and nonce), the operator key, the payout rule exact (whether an owner may leave part of its balance as fee), and a liveness bound $\Delta(N)$ that grows with the number of members.

What the functionality promises. Read as guarantees: (G1) *conservation*, A is always the sum of T and every payout is funded by a removed leaf; (G2) *unilateral exit*, the owner of any leaf, not opaque, is paid its balance within $\Delta(N)$ with no other party's consent; (G3) *eviction*, after the lease height the evictor can do the same; (G4) *non-interactive membership*, JOIN and SWEEP involve only the operator and the ledger; (G5) *no expiry*, no interface depends on the age of the tree. What it does *not* promise is as important: no liveness for joining, and nothing for a member inside an opaque subtree, one the operator grafted without revealing its contents; its value is frozen, not stolen, and §4.4 makes honest members verify their slot before funding a join, so an honest member is never opaque.

The liveness bound. $\Delta(N)$ is not a free parameter. Every call spends and recreates one ledger output, so accepted calls form a chain; if the ledger admits at most κ unconfirmed descendants of an output (25 under today's policy [15], a shape cluster mempool [40] changes but does not remove), an adversary controlling $N-1$ members can delay an honest call by $\lceil N/\kappa \rceil$ confirmations of the tree output, each within the ledger's own bound, giving $\Delta(N) = \Delta_L(\lceil N/\kappa \rceil + 1)$.

Functionality $\mathcal{F}_{\mathsf{LT}}[D, \mathsf{profile}, pk_{\mathrm{op}}, \mathsf{exact}, \Delta]$

State. A tree T of 2^D slots, each *empty*, a leaf $\ell = (a, pk_{\mathrm{exit}}, pk_{\mathrm{own}}, pk_{\mathrm{ev}}, h_{\mathrm{ev}}, nonce, seq)$, or $opaque(s)$: a subtree of sum s never shown. $A = \sum T$ is the amount held. Every call is public (caller, arguments, effect). $\mathcal{A}$ decides when, within $\Delta(N)$ rounds of $\mathcal{G}_{\mathsf{clock}}$, an accepted call takes effect; effects are atomic and ordered.

On $\mathsf{CREATE}(T_0)$ **from** the operator, once: set $T := T_0$ after $\sum T_0$ coins are deposited into $\mathcal{F}_{\mathsf{LT}}$ on $\mathcal{G}_{\mathsf{ledger}}$.
On $\mathsf{JOIN}(p, h, u)$ **from** the operator: require slot p at height h empty and $\mathsf{profile}$ to admit p; require $\sum u$ coins deposited; set $T[p] := u$, where u is a tree of height h, or $opaque(\sum u)$ if $\mathcal{A}$ withholds it.
On $\mathsf{EXIT}(p, v)$ **from** party $\mathcal{P}$: require $T[p] = \ell$ with $\mathcal{P}$ holding pk_{own}, and $v = a$ if exact else $v \le a$; set $T[p] :=$ empty; pay v coins to pk_{exit} on $\mathcal{G}_{\mathsf{ledger}}$.
On $\mathsf{EVICT}(p)$ **from** $\mathcal{P}$: require $T[p] = \ell$, $\mathcal{P}$ holding pk_{ev}, and $\mathcal{G}_{\mathsf{clock}} \ge h_{\mathrm{ev}}$; set $T[p] :=$ empty; pay a to pk_{exit}.
On $\mathsf{UPDATE}(p, \ell')$ **from** the owner of $T[p] = \ell$ and the operator jointly (and the holder of $\ell.pk_{\mathrm{exit}}$ in variant UPDATE^x): set $T[p] := \ell'$; require $\ell'.a - \ell.a$ coins deposited if positive, else pay the difference to the callers.
On $\mathsf{CLAIM}(p, \ell_c, q)$ **from** any party holding a voucher for ℓ_c signed by the owner of $T[p] = \ell$: require $\ell_c.a \le \ell.a$, $\ell_c.seq = \ell.seq$, and $T[q]$ empty once $T[p] := \ell$ with amount $a - \ell_c.a$ and $seq + 1$; then $T[q] := \ell_c$.
On $\mathsf{SWEEP}(p, h)$ **from** $\mathcal{P}$: require every leaf below slot p (height h) to name $\mathcal{P}$ as evictor with $\mathcal{G}_{\mathsf{clock}} \ge h_{\mathrm{ev}}$; set the subtree empty; pay each leaf's a to its pk_{exit}.
On READ **from** any party: return T (with opaque slots as such) and A.

Fig. 1. The ideal functionality. Batched exits (EXIT_K) are K EXIT calls taking effect together. $\mathcal{A}$ may refuse to serve JOIN (there is no liveness for joining), may schedule accepted calls within $\Delta(N)$, and may graft opaque subtrees, which can be swept or replaced but never exited.

Trees are sized by this bound, not by depth: 10,000 channel leaves need about 400 blocks to force-close. The chain of spends also invites fee pinning, which the functionality does not model (§6).

4 The Abstract Protocol

The protocol Π realising $\mathcal{F}_{\mathsf{LT}}$ has three parts: a data structure whose root one output commits to (§4.1, §4.2), one *predicate* per interface over a transaction spending that output, enforced by the ledger (§4.3), and the behaviour of honest parties (§4.4).

4.1 The Tree

We write $\mathsf{le}_k(x)$ for the k-byte little-endian encoding of x. A leaf is a fixed 116-byte record

$$\mathsf{le}_{64}(\mathsf{amount}) \,\|\, \mathsf{exit_key} \,\|\, \mathsf{owner_key} \,\|\, \mathsf{evictor_key} \,\|\, \mathsf{le}_{32}(\mathsf{evict_after}) \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, \mathsf{le}_{32}(\mathsf{seq}),$$

with keys as 32-byte x-only points, $\mathsf{evict_after}$ a block height, nonce a per-leaf identifier and seq a counter advanced by vouchers (Appendix B.2). A *node* is a pair (h, s) of a 32-byte hash and an unsigned sum. A position holding leaf ℓ has

node $(H_{\mathsf{leaf}}(\ell), \ell.\mathsf{amount})$; an empty position has node $E_0 = (0^{32}, 0)$. The parent of (h_L, s_L) and (h_R, s_R) is

$$\big(H_{\mathsf{node}}(h_L \,\|\, \mathsf{enc}_{64}(s_L) \,\|\, h_R \,\|\, \mathsf{enc}_{64}(s_R)),\ s_L + s_R\big), \qquad \mathsf{enc}_{64}(s) = \mathsf{le}_{64}(s) \,\|\, \mathtt{0x01}.$$

Every hash is a BIP-340 tagged hash with its own tag, so no preimage of one is a preimage of another; $\mathsf{enc}_{64}(s)$ is the minimal BIP-441 encoding of $s + 2^{64}$, so a script produces it from a computed sum with one addition (Lemma 1). E_h is the node of the empty tree of height h, hard-coded in scripts.

The *sibling path* π of a position is the list of D pairs $(\mathsf{dir}_k, (h_k, s_k))$, one per level, giving the side the position lies on and the sibling node not taken. $\mathsf{fold}(n, \pi)$ combines a node n with a path level by level from the bottom up, hashing n with each sibling in the order the direction gives and adding the sums. Two facts drive the design: the honest path folds a leaf's node to the root, and folding a *different* node up the *same* path yields the root of the tree with that position replaced (Theorem 1).

Profiles. In the *dense* profile positions are assigned by the operator and a path carries explicit direction bits; each level costs 88 bytes of script and 41 bytes of witness. In the *sparse* profile a leaf's position is the low D bits of $H_{\mathsf{pos}}(\mathsf{owner_key} \,\|\, \mathsf{nonce})$, direction bits are derived in script from the position hash, and empty siblings are elided from the witness (a one-byte marker in place of a node; the script substitutes E_k), so witness data grows with $\log_2 N$ rather than with D. It is the Merkle-sum sparse Merkle tree of Taproot Assets [30] truncated to $D = 32$ or 64, at 1.8 to 2.7 times the dense per-exit cost (Table 6). Dense is the default profile; the sparse profile is retained for deployments that need key-addressed positions, and the rest of the paper measures the dense one unless it says otherwise.

4.2 The Tree Output

Let r be the root hash and S the root sum. The tree output is a taproot output with internal key $P = \mathsf{lift_x}\big(x(\mathsf{NUMS} + H_{\mathsf{state}}(r) \cdot G)\big)$, where NUMS is the BIP-341 nothing-up-my-sleeve point and $\mathsf{lift_x}$ the BIP-340 even-y normalisation, so the key path is unspendable and every spend goes through a script. Its taptree contains one leaf per supported predicate; the taptree's Merkle root m identifies the rule set and is constant for the tree's life. We write $\mathsf{TreeSPK}(r, m)$ for the resulting scriptPubKey; a *tree output* for state (r, S) has script $\mathsf{TreeSPK}(r, m)$ and amount S. Every predicate receives r and m in its witness, recomputes $\mathsf{TreeSPK}(r, m)$ and checks it against the spent input's scriptPubKey, binding both the root the proofs are checked against and the script set the successor carries; without binding m a spender could recreate the tree under scripts of its choosing. Since every predicate checks input 0, two trees cannot be spent atomically.

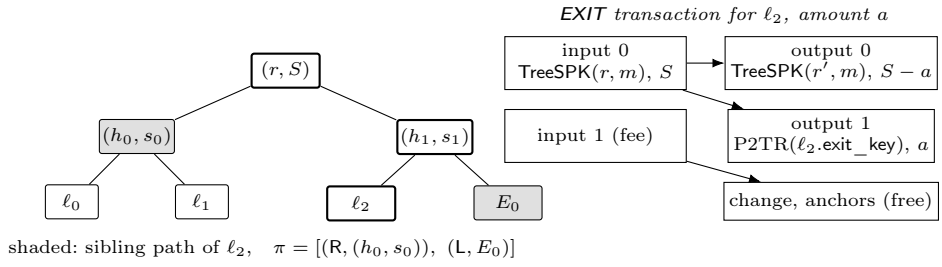


Fig. 2. A depth-2 tree and the exit of ℓ_2. The predicate checks $\mathsf{fold}((H_{\mathsf{leaf}}(\ell_2), a), \pi) = (r, S)$, computes $r' = \mathsf{fold}(E_0, \pi)$, verifies a signature by ℓ_2.owner_key, and constrains outputs 0 and 1. Everything else in the transaction is free.

Table 1. Transition predicates. Each row is one script leaf under m; σ_k is a transaction signature under key k covering all outputs; steps are written old node → new node.

Interface (predicate)	Steps	Authorisation and outputs
EXIT (EXIT)	$(H_{\mathsf{leaf}}(\ell), a) \to E_0$	σ_{own}; out_1 pays exit_key, $v \le a$ ($v = a$ if exact)
EVICT (EXIT, flag)	same	σ_{ev}, lock time $\ge$ evict_after; out_1 pays exactly a
$K\times$EXIT (EXIT$_K$)	K leaf steps, chained	each step its own σ; $\mathrm{out}_{1..K}$
JOIN (JOIN$_h$)	$E_h \to (h_{\mathrm{new}}, s_{\mathrm{new}})$	σ_{op}; $\mathrm{out}_0 = S + s_{\mathrm{new}}$, funded by other inputs
$K\times$JOIN (JOINK$_K$)	K steps $E_0 \to$ leaf	one σ_{op}; the sparse profile cannot graft
UPDATE (UPDATE)	$(H_{\mathsf{leaf}}(\ell), a) \to (H_{\mathsf{leaf}}(\ell'), a')$	$\sigma_{\mathsf{own}}, \sigma_{\mathsf{op}}$, plus σ_{exit} in UPDATE^x; $\mathrm{out}_0 = S - a + a'$
CLAIM (CLAIM)	source → source′; $E_0 \to \ell_c$	voucher signature checked from the stack; $\mathrm{out}_0 = S$
CLAIM (CLAIM^h)	same	as above, over a message naming a payment hash, with the preimage in the witness
CLAIM (SPLIT)	same	σ_{own} of the source leaf; no stack signature, so no `OP_CHECKSIGFROMSTACK`
SWEEP (EVICT_SUBTREE$_h$)	subtree, post-order → E_h	σ_{ev}, every lease expired; one payout per leaf

4.3 The Step and the Transition Predicates

Every transition spends the tree output as input 0 and recreates it as output 0 with the new root and the new total; payouts, if any, occupy outputs $1, \ldots, K$; all other inputs and outputs are unconstrained, so fees, change and anchors are free. The common core is the *step* predicate $\mathsf{Step}(r, S; \pi, n_{\mathrm{old}}, n_{\mathrm{new}}; r', S')$: $\mathsf{fold}(n_{\mathrm{old}}, \pi) = (r, S)$ with S read as input 0's amount (CHECK_OLD), and $(r', S') = \mathsf{fold}(n_{\mathrm{new}}, \pi)$. A transition predicate Pred_X is a sequence of steps, each proving its old node against the root the previous one produced (CHECK_PREV), an authorisation condition, and a check that output 0 is $(S', \mathsf{TreeSPK}(r', m))$ unless $r' = E_D$, in which case the tree is empty and output 0 is free (FINAL). Table 1 gives each interface's predicate; Appendix A has the step as pseudocode and as 88 bytes of tapscript.

Signatures must cover all outputs, or a relayer could redirect the fee slack of a non-exact exit. JOIN$_h$ grafts a whole subtree (with $h = 8$, 256 members) for a constant script cost, but the predicate cannot look inside $(h_{\mathrm{new}}, s_{\mathrm{new}})$:

Table 2. The four opcode tiers. Sizes are bytes of script / witness data for EXIT_1 in the dense profile at $D = 16$; the last row is the transaction size.

	FULL	**MID**	**MIN**	**CCV**
Tapleaf version	`0xC2`	`0xC0`	`0xC0`	`0xC2` or `0xC0`
Beyond BIP-342	`OP_CAT`, splice, bit ops, `OP_TX`, CSFS, EC ops	`OP_CAT`, `OP_TX`, CSFS, `OP_TWEAKADD`	`OP_CAT`, `OP_TWEAKADD`	`OP_CCV` (BIP-443), `OP_TX`
Introspection	`OP_TX`	`OP_TX`	by signature (§5)	`OP_TX`, amounts
Numerics	any length	4-byte, $< 2^{31}$	4-byte, $< 2^{31}$	as FULL or MID
Owner payout	$\leq$ amount	exact	exact	exact
Predicates	all	all but sweeps	EXIT_1, SPLIT	all but vouchers
Executable checks	425	120	31	309
Vouchers/channels/shape	yes / yes / free	yes / yes / free	no / no / fixed	no / yes / free
EXIT_1 script / witness (B)	2,101 / 902	3,299 / 978	3,572 / 1,175	1,716 / 838
EXIT_1 transaction (vB)	1,020	1,338	1,456	908

this is where opaque slots come from. $\mathsf{EVICT_SUBTREE}_h$ is a shape-specialised multiproof, one path to the subtree and its 2^h leaves hashed once rather than once per leaf; a flag-driven generic multiproof was measured and rejected, since without loops it saves no script. Vouchers are optional (Appendix B.2).

4.4 Honest Parties

A member stores its leaf and sibling path and follows every spend of the tree output; since each witness contains the path it used, the member updates its own path from the chain alone and never depends on the operator to exit. Before funding a JOIN, or the creation of a tree, a member verifies its path against the grafted node (or r_0) and the deposited amount; a member who is never added has lost nothing, and one who skips the check may end up opaque. To exit, a member builds the transaction of Figure 2 with a fee input; it treats transitions as final only at its confirmation depth, since a reorganisation invalidates paths derived from dropped ones. The operator maintains the full tree, assigns slots, signs joins and updates, and evicts or sweeps expired leaves.

5 Realisations by Opcode Tier

Each predicate of Table 1 is compiled to tapscript against four opcode sets (Table 2), so that the design's dependence on each candidate opcode is measured rather than asserted; the tree format and the predicates are the same in every tier. Coverage is not uniform: FULL compiles every predicate in both profiles, MID everything but the cohort sweep, MIN the dense single-leaf exit and SPLIT, CCV everything but vouchers. The CCV tier replaces FULL's key recomputation with `OP_CHECKCONTRACTVERIFY` [18] calls; a native Merkle-sum update opcode would absorb the fold as well (hypothetical, §18).

FULL. With `0xC2` the leaf is one 116-byte stack element read with `OP_SUBSTR`; sums use arbitrary-length arithmetic and are padded with $\langle 2^{64} \rangle$ `OP_ADD` (Lemma 1);

one LEVEL of the fold is 88 bytes. TreeSPK is recomputed with the EC opcodes ($H_{\mathsf{state}}(r) \cdot G + \mathsf{NUMS}$, tweaked by the taptree hash, wrapped in a P2TR script); the EXIT_1 prologue, with both introspections and the key recomputation, is under 700 bytes.

MID. Tapleaf `0xC0` keeps BIP-342 semantics: no splice opcodes and 4-byte signed numbers. Fields arrive as separate witness elements, each checked with `OP_SIZE` and re-joined with `OP_CAT` before hashing, and a computed sum is widened to 8 bytes by a witness-supplied zero pad checked with `OP_SIZE` and `OP_CAT` (PADZ, Appendix A); `MINIMALDATA` policy means lock times and positions arrive as minimal numbers plus a pad. A cap of 2^{31} satoshis per tree and exact payouts are the price of 4-byte arithmetic. The tier assumes `OP_TX` in `0xC0`, which the draft leaves open; Liquid [4] has per-field introspection in the same role.

MIN: introspection from a signature. The predicate must read the input's scriptPubKey and amount and outputs 0 and 1. Without `OP_TX` the only opcode that sees the transaction is `OP_CHECKSIG`, and we turn it into introspection. The script *constructs* the BIP-341 signature message from values it has derived (the recomputed $\mathsf{TreeSPK}(r, m)$, the input amount, the new tree output, the payout) and raw witness pieces for everything it does not constrain (the fee input, prevouts, sequences, version, lock time, tapleaf hash), hashes it, and demands a Schnorr signature under the generator G with nonce $R = G$. For public key G and nonce G the challenge is $e = H_{\mathsf{challenge}}(G_x \,\|\, G_x \,\|\, \mathsf{msg})$ and the verification equation $s \cdot G = (1 + e) \cdot G$ has the unique solution $s = e + 1 \bmod n$; G has even y, so the even-y checks pass. The script computes e from its constructed message, forms s, and calls `OP_CHECKSIG` with signature $G_x \,\|\, s$ and key G_x; since `OP_CHECKSIG` hashes the real transaction, the check passes only if the constructed message equals the real one, up to a collision in the tagged hashes. The signature authorises nothing; what it adds is binding, resting on collision resistance of the TapSighash and challenge hashes, outside the Lean model. Without long arithmetic, $e + 1$ is formed by grinding the fee input's sequence number until the last byte of e is `00` (256 trials on average; Appendix A has the enforced fields and the fee-bumping consequences). EXIT_1 costs 3,572 bytes of script at $D = 16$ (70% more than FULL) for 1,456 vB (43% more); vouchers and channel leaves need a signature over stack data and are out of reach, but SPLIT is not (6,441 bytes). The result is an upper bound: BIP-342 tapscript, `OP_CAT` and one point-tweak opcode realise the exit predicate; whether a 64-leaf sweep (16 kB in FULL) fits MIN is open.

CCV: binding by opcode. `OP_CHECKCONTRACTVERIFY` [18] replaces every taproot-key recomputation of FULL with a native check: one call binds the old root to the input under the input's own taptree, one commits output 0 to the new root with the residual assigned, one per payout asserts a bare P2TR of the leaf's exit key and deducts its amount. The state key becomes the BIP-443 data tweak of NUMS by the root, so the tier is a tree-creation parameter. The fold and the authorisation are FULL's unchanged; the EC-ops draft drops out and r and m

leave every witness; the binding is independent of numerics, so the tier also comes with MID's `0xC0` encoding. One `OP_TX` read per constrained amount survives, so the tier takes MID/MIN's exact-payout rule with exogenous fees; vouchers and CLAIM^h, whose message binds m, are not carried; SPLIT, whose transaction signature names no m, is. Measured at dense $D = 16$: EXIT_1 1,716 / 838 B of script / witness against FULL's 2,101 / 902.

6 Security

We prove that Π realises $\mathcal{F}_{\mathsf{LT}}$ in two layers. The tree-level facts (binding, conservation, payability) are theorems about the abstract predicates, mechanised in Lean 4 [24] in about 1,300 lines without Mathlib (§8) and cited by their statements. The composable claim is a simulation argument in the hybrid model of §2, sketched here and given in §7. We work in the idealised hash model: H_{leaf} and H_{node} are injective on the domains the protocol uses, and digests under different tags never coincide or equal the zero string; a proof under injectivity holds against any adversary that cannot exhibit a collision. Predicates and bytecode correspond by execution (§11), not proof.

6.1 Tree-Level Lemmas

Lemma 1 (Encodings). *For $s < 2^{64}$, the minimal BIP-441 encoding of $s + 2^{64}$ is $\mathsf{le}_{64}(s) \,\|\, \mathtt{01}$; leaf and node serialisations are injective under the field bounds; the minimal encoding of 2^{32} has five bytes while that of $2^{32} - 1$ has four; and the `CScriptNum` of $-(2^{31} - 1)$ is `ffffffff` with successor `feffffff`. (enc64_eq_minimal, serializeLeaf_inj, serializeNode_inj, minimalLE_two_pow_32_length, scriptNum_neg_max31_succ.)*

Theorem 1 (Binding). *Roots bind trees: two trees of the same depth with equal roots are equal. Any node n and path π with $\mathsf{fold}(n, \pi)$ equal to the root of a tree T are the root of one of T's subtrees and that subtree's honest path; in particular the sum in n is the true sum of that subtree, and a leaf node that verifies with a full-length path is the leaf at that position.*

Proof (sketch). Induction on the path. A non-empty path claims an internal node, so the tree is not a tip by tag separation. Injectivity of H_{node} on the parent's preimage equates both children's hashes *and* sums with the claimed ones, and the induction hypothesis applies to the child the direction bit selects. This is the only place the hash axioms are used.

Theorem 2 (Conservation). *For any sequence of valid steps from (r, S) to (r', S'), $S' + \sum s_{\text{old}} = S + \sum s_{\text{new}}$. Hence for EXIT_K, $S' + \sum_k \text{payout}_k \le S$, with equality under exact payouts; for JOIN_h, $S' = S + s_{\text{new}}$; for UPDATE, $S' + a_{\text{old}} = S + a_{\text{new}}$; and CLAIM leaves S unchanged. No hash assumption is needed: the predicate conserves sums whatever nodes it is shown.*

Theorem 3 (Payability). *For any tree T and position p holding leaf ℓ, the honest witness (the leaf, its path, E_0) is a valid step from T's root to the root of T with p emptied, the sum falls by $\ell.\mathsf{amount}$, and every other position is unchanged.*

Theorem 4 (Real-tree invariant). *If the current root is the root of a tree and every step is well formed and* grounded *(its new node is the root of some tree of the step's height), the final root is the root of a tree of the same depth, namely the initial tree with each step's subtree replaced. Leaf-level steps are grounded by construction; a JOIN graft is grounded exactly when the operator's node has a preimage tree.*

update_sound and claim_sound locate the positions of UPDATE and CLAIM the same way; mechanising CLAIM showed that the claim position need only be empty *after* the source step, which is harmless. Authorisation and lock times decide who may perform a transition, not what it does; the theorems carry them as an opaque proposition auth, discharged by $\mathcal{F}_{\mathsf{sig}}$ and $\mathcal{G}_{\mathsf{clock}}$ in the simulation.

Root binding to the output. The predicates conclude that the witness root and taptree are the committed ones from $\mathsf{TreeSPK}(r', m') = \mathsf{TreeSPK}(r, m)$, which needs TreeSPK injective in (r, m) (witness_root_binds); §8 reduces this to the discrete logarithm of NUMS and collisions in H_{state} or the taproot tweak hash.

6.2 The Composable Statement

Let Π_τ be the protocol of §4 with the predicates compiled by tier τ, and let $\mathcal{G}_{\mathsf{ledger}}[\mathsf{Pred}_\tau]$ be the global ledger whose validity predicate, for a spend of a tree output, is that tier's predicate. Let $\mathcal{F}_{\mathsf{LT}}|_\tau$ be the functionality restricted to the interfaces τ realises (Table 2), with exact forced in MID, MIN and CCV.

Theorem 5 (UC realisation). *In the idealised hash model, for each tier τ and any D and profile, Π_τ UC-realises $\mathcal{F}_{\mathsf{LT}}|_\tau[D, \mathsf{profile}, pk_{\mathrm{op}}, \mathsf{exact}, \Delta]$ in the hybrid model with $\mathcal{G}_{\mathsf{ledger}}[\mathsf{Pred}_\tau]$, $\mathcal{G}_{\mathsf{clock}}$ and $\mathcal{F}_{\mathsf{sig}}$, against a static adversary corrupting the operator and any subset of members, with $\Delta(N) = \Delta_L(\lceil N/\kappa \rceil + 1)$, provided the tree is created with a real tree T_0 whose sum is the funded amount.*

Proof (sketch). The simulator $\mathcal{S}$ runs the honest parties internally and translates between ledger spends and functionality calls. For every spend of the tree output that $\mathcal{G}_{\mathsf{ledger}}$ accepts, $\mathcal{S}$ parses the witness, folds each step's old node up its path, and by Theorem 1 recovers the unique slot it proves (or aborts on a collision, an event the hash model excludes); it then calls the matching interface of Figure 1 on the authoriser's behalf, checking the signatures with $\mathcal{F}_{\mathsf{sig}}$ (a signature by an honest key the honest party did not issue is a forgery, excluded by $\mathcal{F}_{\mathsf{sig}}$). Theorem 4 gives that the functionality's tree after the call has exactly the root the transaction committed, Theorem 2 that the amounts the functionality pays are the transaction's outputs, so the ledger contents agree in both worlds. Conversely, when $\mathcal{F}_{\mathsf{LT}}$ executes an honest EXIT or EVICT, Theorem 3 lets $\mathcal{S}$ build an accepted real transaction from the honest party's leaf and path, which it submits within

$\Delta(N)$; the contention bound of §3 is what makes $\Delta(N)$ sufficient. A graft whose preimage $\mathcal{S}$ cannot obtain becomes an opaque slot in $\mathcal{F}_{\mathsf{LT}}$, matching Theorem 4's conditionality. Creation is the base case the covenant cannot enforce, hence the proviso. The hybrids and the per-tier remarks (MIN's binding rests additionally on collision resistance of the TapSighash) are in §7.

What the theorem leaves out. The predicates are matched to the bytecode by 885 executable checks (§11), not by a verified compiler; the theorem is about Pred_τ as specified. Fee pinning is outside the ledger model: a low-feerate transition early in the chain of unconfirmed spends caps the feerate of everything behind it, and replacing it means paying for its descendants; mitigations are cooperative batching, anchors so either party can bump, and sizing trees by exit throughput. This is the design's honest weak spot. The operator can evict any leaf after its lease, and a cohort at about 160 vB per leaf, which makes griefing cheap; the functionality states this as a capability, and members who want a hard guarantee buy it with a future lease height.

7 The UC Proof

This section gives the simulator and the hybrid argument behind Theorem 5. We follow the conventions of the composable ledger treatment of Badertscher et al. [2] and of Generalized Channels [1]: the ideal functionality itself has access to the global ledger and posts the transactions that move coins, so that the ideal and real worlds agree on what the ledger contains and differ only in how the tree state is maintained.

7.1 The Hybrid World

The real protocol Π_τ runs in the presence of:

$\mathcal{G}_{\mathsf{ledger}}[\mathsf{Pred}_\tau]$ the global ledger functionality of Badertscher et al., parameterised by a validity predicate that, for a transaction spending a tree output, evaluates the tier's covenant predicate Pred_τ on the input state, the witness and the outputs (§4). It guarantees persistence, and liveness with parameter Δ_L: a valid transaction with sufficient fee that an honest party submits is in the ledger state of every honest party within Δ_L rounds. The adversary chooses the order of transactions within the rules and may delay a submitted transaction by up to Δ_L.

$\mathcal{G}_{\mathsf{clock}}$ the global clock of Katz et al. [19]; rounds are blocks.

$\mathcal{F}_{\mathsf{sig}}$ the ideal signature functionality [7], which we use in place of an EUF-CMA reduction; every `CHECKSIG` and `CHECKSIGFROMSTACK` in the scripts is a call to its verification interface.

Hashes are treated in the idealised model of §6 (injectivity plus tag separation); the simulator's abort events are exactly the collision events this model excludes, so the argument transfers to a collision-resistance reduction in the standard way.

7.2 The Simulator

$\mathcal{S}$ runs an internal copy of every honest party's protocol state and of the adversary $\mathcal{A}$, and interposes between the environment, the ideal functionality $\mathcal{F}_{\mathsf{LT}}$ and the hybrids. Its behaviour is driven by two kinds of events.

Ledger events. Whenever a transaction tx spending a tree output appears in the ledger (honest or adversarial), $\mathcal{S}$ parses its witness and translates it into $\mathcal{F}_{\mathsf{LT}}$ calls:

1. Identify the script leaf executed from the control block; this fixes the transition type and its parameters (K, h, profile, variant).
2. For each step in the witness, run the fold on the supplied path and node. Because tx is valid, the old node folds to the committed root (CHECK_OLD or CHECK_PREV). By Theorem 1 (fold_sound), $\mathcal{S}$ recovers the unique position p of that node in its copy of the tree T, or detects that no such position exists, which is a collision (abort event Coll).
3. Map the step to the interface: a leaf node replaced by E_0 with the owner-flag clear is EXIT(p, *payout*), with the flag set EVICT(p); E_h replaced by a node is JOIN(p, u) for a subtree u that $\mathcal{S}$ obtains from the adversary's join messages (or records as ungrounded, see below); a leaf replaced by a leaf is UPDATE; the two-step pattern of §4 is CLAIM; the post-order pattern is SWEEP.
4. Verify with $\mathcal{F}_{\mathsf{sig}}$ that the authorising signatures are valid for the keys the leaf names. A valid signature for an honest party's key that the honest party did not produce is a forgery (abort event Forge), which $\mathcal{F}_{\mathsf{sig}}$ makes impossible.
5. Invoke the interface on $\mathcal{F}_{\mathsf{LT}}$ on behalf of the party that authorised the transition (the corrupt party, or, for honest parties' own transactions, by letting the honest party's ideal instruction through).

Functionality events. When $\mathcal{F}_{\mathsf{LT}}$ executes an interface for an honest party, $\mathcal{S}$ constructs the corresponding real transaction from its copy of the honest state (leaf, path, signature via $\mathcal{F}_{\mathsf{sig}}$) and submits it to $\mathcal{G}_{\mathsf{ledger}}$ on the honest party's behalf, scheduling it within the delay $\mathcal{F}_{\mathsf{LT}}$ permits. $\mathcal{F}_{\mathsf{LT}}$'s payouts are the ledger outputs of that transaction, so the two worlds agree on the ledger.

Ungrounded grafts. If a corrupt operator grafts a node whose preimage tree $\mathcal{S}$ cannot obtain, $\mathcal{S}$ records the slot as *opaque* in $\mathcal{F}_{\mathsf{LT}}$ (the functionality's JOIN accepts an opaque node of the stated sum). Opaque slots contribute their sum to A and can be swept or replaced but never exited; this is the real behaviour (Theorem 4), and an honest member never funds a join into an opaque slot because Π verifies its own path before funding.

7.3 Hybrids

$\mathcal{H}_0$ the real execution of Π_τ with $\mathcal{A}$ in the $(\mathcal{G}_{\mathsf{ledger}}, \mathcal{G}_{\mathsf{clock}}, \mathcal{F}_{\mathsf{sig}})$-hybrid world.
$\mathcal{H}_1$ as $\mathcal{H}_0$, but the execution aborts on Forge. Indistinguishable by the definition of $\mathcal{F}_{\mathsf{sig}}$.

$\mathcal{H}_2$ as $\mathcal{H}_1$, but the execution aborts on Coll: a valid witness whose old node is not the node at any position of the honest tree. By Theorem 1, in the idealised hash model Coll has probability zero; under collision resistance it yields two preimages of H_{node} or H_{leaf} with equal digests.

$\mathcal{H}_3$ as $\mathcal{H}_2$, but the honest parties' tree state is replaced by $\mathcal{F}_{\mathsf{LT}}$'s and their transactions are produced by $\mathcal{S}$ from functionality events. We argue the two are identical transaction by transaction: for every accepted spend, step 2 of $\mathcal{S}$ finds the position and Theorem 4 (Step.sound) gives that the new root is the root of T with that subtree replaced, so $\mathcal{F}_{\mathsf{LT}}$'s state after the interface call equals the honest parties' real state. Conservation (Steps.sum) gives that the output amounts $\mathcal{F}_{\mathsf{LT}}$ pays are those the transaction carries; the real-tree invariant (Steps.sound) gives that $\mathcal{F}_{\mathsf{LT}}$'s T is a tree whenever the real root is; and payability (payable) gives that whenever $\mathcal{F}_{\mathsf{LT}}$ executes an honest EXIT, $\mathcal{S}$ can produce a valid real transaction. The ledger and clock are shared, so the environment's view of them is unchanged.

$\mathcal{H}_4$ the ideal execution of $\mathcal{F}_{\mathsf{LT}}$ with $\mathcal{S}$. Identical to $\mathcal{H}_3$ by construction.

Liveness. $\mathcal{F}_{\mathsf{LT}}$ permits $\mathcal{S}$ to delay an honest EXIT or EVICT by up to $\Delta(N)$. In the real world an honest exit competes for the tree output with at most $N-1$ other transitions, of which at most κ (the ancestor limit, 25 transactions per unconfirmed chain) can be pending per confirmed output; an adversary who front-runs maximally therefore delays the honest transaction by at most $\lceil N/\kappa \rceil$ confirmations of the tree output, each within Δ_L, giving $\Delta(N) = \Delta_L \cdot (\lceil N/\kappa \rceil + 1)$. This bound assumes the honest party pays a fee sufficient for $\mathcal{G}_{\mathsf{ledger}}$'s liveness guarantee; the pinning discussion of §6 is what happens when it does not.

7.4 Per-Tier Remarks

The argument is the same for every tier because the predicates are the same; what differs is which predicates the tier realises (Table 2) and one detail each of the MIN and CCV tiers. In MIN, CHECK_OLD and FINAL are enforced through the constructive signature check of §5, whose binding rests on collision resistance of the TapSighash and challenge hashes rather than on $\mathcal{G}_{\mathsf{ledger}}$'s predicate evaluation; $\mathcal{H}_2$ therefore also aborts on a collision in those hashes, and $\mathcal{S}$ recovers the enforced fields from the witness's raw pieces rather than from `OP_TX` selectors. In CCV, the witness carries neither r nor m: the input's commitment to the old root and output 0's commitment to the new one are checked by `OP_CHECKCONTRACTVERIFY` under the input's own taptree, so the step of $\mathcal{S}$ that reads r and m from the witness reads them from the spent output instead, and the binding of witness_root_binds rests on the BIP-443 data tweak in place of TreeSPK, with the same reduction to the discrete logarithm of NUMS and collisions in the tweak hash. Its interfaces are FULL's, with CLAIM only in its SPLIT form, under exact payouts.

7.5 Composition for Channel Leaves

Theorem 6 follows the pattern of Generalized Channels [1], in the hybrid model with $\mathcal{F}_{\mathsf{LT}}$, $\mathcal{G}_{\mathsf{ledger}}$, $\mathcal{G}_{\mathsf{clock}}$ and $\mathcal{F}_{\mathsf{sig}}$. The channel simulator reuses the channel-level

simulator of that work for the update and settlement transactions, which are ordinary ledger transactions once the funding output exists, and uses $\mathcal{F}_{\mathsf{LT}}$'s EXIT and EVICT interfaces to materialise it. The three tree-layer preconditions of §9 are exactly the conditions under which $\mathcal{F}_{\mathsf{LT}}$ guarantees that the materialised output carries the full capacity to the channel key: exact payouts (ExitK.exact_conservation), non-claimability (the SEQ_FINAL facts), and a lease at or below the tip so that EVICT is available to the counterparty. We give this as a sketch and leave the full channel-level proof, in particular the HTLC deadline argument under $\Delta(N)$, to future work.

8 The Lean Formalisation

This section describes the mechanised model behind §6 in enough detail that a reader can judge what has been proved, under what assumptions, and what has not. The development is about 1,300 lines of Lean 4 [24] in seven modules and depends on nothing but the core library: no Mathlib, no external tactics. It builds in seconds and prints an axiom audit for every headline theorem; each depends only on propext, Quot.sound and, in one case, Classical.choice, and none on sorryAx. The code is in the artefact [32].

8.1 Design of the Model

The model states the *rules* of §4.3 as relations over the objects the scripts manipulate: nodes, paths, amounts. It does not model tapscript bytecode, signatures, keys or lock times. Authorisation is carried as an opaque proposition, and the input and output amounts the scripts read with OP_TX are parameters. Three choices keep the proofs short.

Depth as a type index. A tree of depth d is Tree d, a perfect binary tree whose tips are Option Leaf. Positions are Pos d h, paths from depth d down to a subtree of height h, so that Pos d 0 addresses a leaf and Pos d h addresses the slot a JOIN_h fills. Indexing by both endpoints avoids all arithmetic on depths: the recursions on getSub, setSub and pathSub pattern-match on the position and the tree simultaneously, and the one impossible case (a cons step below height 0) is dismissed by Pos.height_le.

```
inductive Tree : Nat → Type
  | tip : Option Leaf → Tree 0
  | bin {d : Nat} : Tree d → Tree d → Tree (d + 1)

inductive Pos : Nat → Nat → Type
  | nil {h : Nat} : Pos h h
  | cons {d h : Nat} : Bool → Pos d h → Pos (d + 1) h

def parent (a b : Node H) : Node H :=
  ⟨M.hNode a.hash a.sum b.hash b.sum, a.sum + b.sum⟩

def combine (n : Node H) (x : Bool × Node H) : Node H :=
  if x.1 then parent x.2 n else parent n x.2
```

```
def fold (n : Node H) : List (Bool × Node H) → Node H
  | [] => n
  | x :: rest => combine (fold n rest) x
```

Root-first paths. The model orders a sibling path from the root down, so fold is a structural recursion on the list and the soundness proof is a plain induction. The witness of §5 supplies the same pairs leaf-first, because the script consumes the deepest level first; nothing in the theorems depends on the order.

Hashes as parameters. Everything is parametric in a class HashModel whose fields are the two hash functions, the zero digest, and the assumptions the argument uses:

```
class HashModel (Leaf : Type) (H : outParam Type) where
  hLeaf : Leaf → H
  hNode : H → Nat → H → Nat → H
  empty0 : H
  hLeaf_inj : ∀ {l₁ l₂}, hLeaf l₁ = hLeaf l₂ → l₁ = l₂
  hNode_inj : ∀ {a₁ b₁ a₂ b₂ sa₁ sb₁ sa₂ sb₂},
    hNode a₁ sa₁ b₁ sb₁ = hNode a₂ sa₂ b₂ sb₂ →
      a₁ = a₂ ∧ sa₁ = sa₂ ∧ b₁ = b₂ ∧ sb₁ = sb₂
  hNode_ne_hLeaf : ∀ a sa b sb l, hNode a sa b sb ≠ hLeaf l
  hNode_ne_empty : ∀ a sa b sb, hNode a sa b sb ≠ empty0
  hLeaf_ne_empty : ∀ l, hLeaf l ≠ empty0
```

Injectivity is collision resistance idealised; the three inequalities are the tag separation that distinct BIP-340 tags provide. The encodings module discharges the serialisation half of each injectivity field (Lemma 1): given SHA-256 collision resistance, serializeLeaf_inj and serializeNode_inj are what remains. Nothing quantifies the adversary's advantage; a proof under injectivity transfers to any adversary that cannot exhibit a collision.

8.2 Theorem Statements

The statements below are verbatim from the development. t.root is the root node of a tree, t.getSub p the subtree at position p, t.setSub p u the tree with that subtree replaced by u, t.pathSub p the honest sibling path, and t.get, t.set, t.path their leaf-level specialisations.

```
theorem fold_pathSub (t : Tree d) (p : Pos d h) :
    fold (t.getSub p).root (t.pathSub p) = t.root

theorem fold_setSub (t : Tree d) (p : Pos d h) (s : Tree h) :
    fold s.root (t.pathSub p) = (t.setSub p s).root

theorem root_inj (t₁ t₂ : Tree d) : t₁.root = t₂.root → t₁ = t₂

theorem fold_sound (t : Tree d) (m : Node H) (π : List (Bool × Node H)) :
    fold m π = t.root →
    ∃ (h : Nat) (p : Pos d h), m = (t.getSub p).root ∧ π = t.pathSub p
```

fold_sound is Theorem 1. Its proof is the only place the hash axioms are used: a non-empty path claims an internal node, so the tree is not a tip by tag separation, and injectivity of hNode equates both children's hashes and sums with the claimed ones. Note that the conclusion equates *nodes*, so the sum the prover presented is the true sum of the subtree.

A Step is a path, a height, an old node and a new node; it is valid for roots r, r' when the old node folds to r and the new node to r', and well formed for depth d when the path has $d - h$ entries. Steps chains steps, each against the root the previous one produced.

```
theorem Step.sound (t : Tree d) (s : Step H) (r' : Node H)
    (hwf : s.WF d) (hv : s.valid t.root r') :
    ∃ p : Pos d s.height, s.old = (t.getSub p).root ∧
      ∀ (u : Tree s.height), u.root = s.new → r' = (t.setSub p u).root

theorem Steps.sum (h : Steps r ss r') :
    r'.sum + oldSum ss = r.sum + newSum ss

theorem Steps.sound (t : Tree d) (h : Steps t.root ss r')
    (hwf : ∀ s ∈ ss, s.WF d) (hg : ∀ s ∈ ss, s.Grounded) :
    ∃ t' : Tree d, t'.root = r'
```

Steps.sum is Theorem 2 and needs no hash assumption. Steps.sound is Theorem 4; s.Grounded says the new node is the root of some tree of the step's height, which leaf-level steps satisfy by construction (grounded_of_leaf) and which a JOIN graft must be given.

The transitions are structures bundling the steps with the output rule and an opaque authorisation. Their theorems:

```
theorem payable (t : Tree d) (p : Pos d 0) (l : Leaf) (hl : t.get p = some l) :
    (leafStep (t.path p) (some l) none).valid t.root (t.set p none).root ∧
    (t.set p none).root.sum + l.amount = t.root.sum ∧
    ∀ q, q ≠ p → (t.set p none).get q = t.get q

theorem ExitK.no_inflation (h : ExitK exact r blocks r' out0) :
    out0 + payoutSum blocks ≤ r.sum

theorem ExitK.exact_conservation (h : ExitK true r blocks r' out0) :
    out0 + payoutSum blocks = r.sum

theorem join_sound (t : Tree d) (hwf : path.length + h = d)
    (hj : Join auth h t.root path new r' out0) :
    ∃ p : Pos d h, t.getSub p = Tree.empty h ∧
      out0 = t.root.sum + new.sum ∧
      ∀ u : Tree h, u.root = new → r' = (t.setSub p u).root

theorem claim_sound (t : Tree d) (hwf₁ : srcPath.length = d)
    (hwf₂ : claimPath.length = d)
    (hc : Claim auth t.root srcPath claimPath src claim r' out0) :
    out0 = t.root.sum ∧
    ∃ (p₁ p₂ : Pos d 0), t.get p₁ = some src ∧
      (t.set p₁ (claimedSource src claim.amount)).get p₂ = none ∧
      r' = ((t.set p₁ (claimedSource src claim.amount))
              .set p₂ (some claim)).root
```

payable is Theorem 3. join_sound uses root_inj (via eq_empty_of_root): since JOIN_h hard-codes E_h as the old node, the slot it fills is provably the empty tree, and the theorem gives the successor root for every tree whose root is the grafted node. claim_sound exposes the drained-source case: the claim position must be empty in the tree *after* the source step, which is why a source reduced to zero may host the claim.

8.3 Assumption Ledger

Every hypothesis outside Lean's logic, what it stands for in the deployment, and which theorems consume it.

hypothesis	stands for	used by
hLeaf_inj	SHA-256 collision resistance + serialization	leafNode_inj, root_inj
hNode_inj	SHA-256 collision resistance + serialization	root_inj, fold_sound
hNode_ne_hLeaf, hNode_ne_empty, hLeaf_ne_empty	distinct tags (random-oracle heuristic)	leafNode_inj, fold_sound
treeSpk_inj	DL on secp256k1 + CR of H_{state}, TapTweak	witness_root_binds
real-tree invariant	creation check + grounded grafts (off-chain)	every theorem stated for a Tree
(none)		Steps.sum, ExitK.no_inflation, encodings

The conservation results use no hypothesis at all, which is the formal content of "the covenant conserves sums whatever it is shown". The collision-resistance assumption on the TapSighash and challenge hashes that the MIN tier's constructive introspection adds (§5) is outside the model, since the model does not include signature checking.

8.4 Modules and Effort

module	lines	contents
Hash.lean	59	the HashModel class
Leaf.lean	96	Leaf, Leaf.WF, Node, leafNode, leafNode_inj
Tree.lean	335	trees, positions, paths, fold, all tree theorems
Transitions.lean	458	Step, Steps, the transitions and their theorems
Binding.lean	60	BindingModel and the two binding theorems
Encoding.lean	238	encodings, serialisation injectivity, SEQ_FINAL
Axioms.lean	32	the axiom audit

The only proofs that took real effort were fold_sound, where the two-index positions and the domain-separation case for a tip had to be found, and Steps.sound, where the height of the grounding tree must be reconciled with the height the soundness lemma returns, which is why Step carries its height explicitly. A verified interpreter for the tapscript subset would need a model of the stack machine, of BIP-441 numerics, of `OP_TX`'s serialisations and of the BIP-341 message, and a proof that each generated script's execution relation equals the corresponding Step relation; we estimate it at several thousand lines and regard it as the natural sequel.

8.5 Binding to the Output

Binding.lean records the one cryptographic step outside the hash model. A structure BindingModel carries an abstract treeSpk : H → TapRoot → Spk with the hypothesis that it is injective in both arguments, and two one-line theorems conclude that a witness (r', m') passing the scriptPubKey check equals the committed (r, m), and that the successor output commits to the computed root under the same m. In the deployment the map is $(r, m) \mapsto x\big(P_r + H_{\mathsf{TapTweak}}(x(P_r) \,\|\, m) \cdot G\big)$ with

$P_r = \mathsf{lift_x}(x(\mathsf{NUMS} + H_{\mathsf{state}}(r)G))$, and the two x-projections identify a point with its negation. A collision $(r, m) \neq (r', m')$ yields one of: $x(P_r) = x(P_{r'})$ with $r \neq r'$, which is a collision in H_{state} or the relation $\mathsf{NUMS} + tG = -(\mathsf{NUMS} + t'G)$, solvable for the discrete logarithm of NUMS; $x(P_r) = x(P_{r'})$ with $m \neq m'$, a collision in H_{TapTweak}; or distinct internal keys with $Q = \pm Q'$, again a linear relation in NUMS and G solvable for the discrete logarithm of NUMS. Making the hypothesis explicit is the point: it is what a future script change must not break.

8.6 Encodings

Encoding.lean proves the byte-level facts the design leans on, over natural-number bytes so that every proof stays within linear arithmetic: fixed-width little-endian encodings are injective on their range; `minimalLE (n + 2^(8k)) = leBytes k n ++ [1]` for $n < 2^{8k}$, which with $k = 8$ is enc_{64} and justifies producing it with one addition; the 116-byte leaf and the 82-byte node preimage are injective; and the SEQ_FINAL facts of Lemma 1, computed by `decide` on a bounded-recursion model of `CScriptNum`.

8.7 What Is Not Proved

The tapscript bytecode is outside the model. That the scripts of §5 implement the relations above is established by execution: every script of every tier runs in the simulator against a reference tree with positive and negative cases, and the Go generators are compared byte for byte with the Python output. A verified interpreter for the tapscript subset (BIP-441 numerics, `OP_TX`, `OP_CHECKSIGFROMSTACK`) would close this gap and is a project of a different size. Signatures, lock times and the transaction structure beyond amounts are authorisation-side facts and are deliberately opaque. Concurrency, mempool policy, reorganisations and the off-chain protocol are not theorems about hash trees and belong to a model checker; a design for a P [13] model is in the artefact.

9 Channel Leaves as Composition

A leaf whose exit key is a 2-of-2 between its owner and a counterparty (normally the operator) can host a payment channel that exists only off-chain and whose funding output materialises at the payout of the EXIT or EVICT that removes the leaf: a tree of N such leaves is a Lightning hub behind one UTXO, with opening, resizing and closing costing nothing on-chain except at dispute. In UC terms the channel protocol runs in the $\mathcal{F}_{\mathsf{LT}}$-hybrid model: it treats the functionality's payout as its funding transaction. §10 gives scripts, state machine and costs.

Rebindable commitments. The funding outpoint is unknown when channel states are signed, and it changes with every tree transition, so commitment transactions are authorised by signatures that do not commit to the prevout: `OP_TX`

collates the fields the signature covers (version, lock time, the current input's sequence, the number of inputs, all outputs) behind a per-channel context constant, and `OP_CHECKSIGFROMSTACK` verifies the channel key's signature over the hash, emulating `SIGHASH_ANYPREVOUT` [12]. The context commits to m, the leaf identity, the channel key, the capacity and a splice epoch, so a state signed for one channel or capacity never verifies for another. Penalty transactions cannot be presigned against an unknown outpoint, so channel leaves use the update/settlement ratchet of eltoo [11,25], with zero-fee version-3 transactions and an ephemeral anchor [44]. Settlement outputs include one per in-flight HTLC; the deadline budget is $\mathsf{cltv_expiry_delta} \geq G_{\text{exit}} + 3 + \mathsf{csv_delay} + G_{\text{sweep}}$, where G_{exit} is the time to get the exit confirmed including contention, that is, $\Delta(N)$. With $\mathsf{csv_delay} = 144$ and $G_{\text{exit}} = 72$ this is about 220 blocks, against 40 to 80 on the public network.

Composition conditions. Let $\mathcal{F}_{\mathsf{chan}}$ be a channel functionality in the style of Generalized Channels [1], whose CLOSE pays the latest balances on the ledger within a bound. Three of $\mathcal{F}_{\mathsf{LT}}$'s interfaces are too lenient for a leaf whose balance is not the owner's alone, and each becomes a precondition. (i) A non-exact EXIT would let the owner underfund the channel and hold the counterparty's balance hostage, so channel leaves live in trees with exact set, visible in m. (ii) CLAIM lets the owner carve value out of a leaf alone, so channel leaves set seq to the value whose successor does not fit in four bytes: `0xFFFFFFFF` under BIP-441 numerics but `0x7FFFFFFF` under `CScriptNum` (Lemma 1). (iii) Before the lease height the counterparty can enforce nothing on-chain, so a routing channel leaf sets it to zero, giving the counterparty EVICT; between two members, UPDATE^x adds the exit key to UPDATE's authorisers.

Theorem 6 (Composition, sketch). *In the* $(\mathcal{F}_{\mathsf{LT}}, \mathcal{G}_{\mathsf{ledger}}, \mathcal{G}_{\mathsf{clock}}, \mathcal{F}_{\mathsf{sig}})$*-hybrid model with conditions (i) to (iii), the channel-leaf protocol of §10 realises* $\mathcal{F}_{\mathsf{chan}}$ *with closing bound* $\Delta(N) + 3 + \mathsf{csv_delay}$*, for channels whose HTLC deadlines respect the budget above.*

The argument (§7) reuses the channel-level simulator of [1] and uses EXIT and EVICT to materialise the funding output; the HTLC deadline argument under $\Delta(N)$ is future work. Channel leaves need `OP_TX` and `OP_CHECKSIGFROMSTACK`, hence FULL or MID, or CCV with the channel scripts in their own outputs; MIN's constructive introspection commits to the prevout.

10 Channel Leaves: Scripts and Costs

Selectors and context. Two `OP_TX` selectors are used, both with the collate flag so that one call leaves one blob: SEL_CHAN_UPD selects version, number of inputs, the current input's sequence, number of outputs, all output amounts and scripts, and lock time; SEL_CHAN_SET adds the current input's amount and

REBIND_n (tapleaf 0xC2, 81 B)	SETTLE_n (tapleaf 0xC2, 80 B)
$\langle S_0 + n + 1 \rangle$ `OP_CHECKLOCKTIMEVERIFY OP_DROP`	$\langle \mathsf{csv_delay} \rangle$ `OP_CHECKSEQUENCEVERIFY OP_DROP`
$\langle \mathsf{SEL_CHAN_UPD} \rangle$ `OP_TX` // collated blob	$\langle \mathsf{SEL_CHAN_SET} \rangle$ `OP_TX` // + input amount, spk
$\langle \mathsf{ctx_{upd}} \rangle$ `OP_SWAP OP_CAT OP_SHA256`	$\langle \mathsf{ctx_{set}} \rangle$ `OP_SWAP OP_CAT OP_SHA256`
$\langle P_{\mathsf{chan}} \rangle$ `OP_CHECKSIGFROMSTACK`	$\langle P_{\mathsf{chan}} \rangle$ `OP_CHECKSIGFROMSTACK`

Fig. 3. The two leaves of a channel leaf's update output for state n.

scriptPubKey. The per-channel context is

$$\mathsf{chan_id} = H_{\mathsf{chanid}}(m \,\|\, \mathsf{owner_key} \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, x(P_{\mathsf{chan}}) \,\|\, \mathsf{le}_{64}(C) \,\|\, \mathsf{le}_{32}(\mathsf{epoch}) \,\|\, \mathsf{le}_{32}(\mathsf{csv_delay})),$$

with $\mathsf{ctx_{upd}}$ and $\mathsf{ctx_{set}}$ two tagged hashes of it, so that an update signature never verifies where a settlement signature is required. The signed messages are $\mathsf{SHA256}(\mathsf{ctx} \,\|\, \mathtt{OP_TX}(\mathsf{SEL}))$.

Update output scripts. The update transaction for state n pays its whole input value to an output U_n with internal key P_{chan} (a MuSig2 [27,26] key of the two parties) and the two-leaf taptree of Figure 3. A later update binds to U_n through REBIND_n, whose lock-time check admits only states above n; the settlement of state n binds through SETTLE_n after $\mathsf{csv_delay}$ blocks, and only to U_n because $\mathsf{SEL_CHAN_SET}$ commits the spent script, which embeds n.

Transactions. The update transaction is version 3, lock time $S_0 + n$, one input with sequence `0xFFFFFFFD`, one output of the full capacity C to U_n and a zero-value pay-to-anchor output; it pays no fee and is valid whether bound to the funding output or to any earlier update output, both of value C. The settlement is version 3 with input sequence $\mathsf{csv_delay}$, outputs for the two balances and each in-flight HTLC, and an anchor; $b_U + b_V + \sum \mathrm{htlc} = C$. Fees for both come from a child spending the anchor, so both parties need an on-chain fee reserve to close unilaterally. The signing order is settlement before update for each state, as in eltoo.

Force-close cost. At $D = 16$ dense, $\mathsf{csv_delay} = 144$ and no HTLCs in flight:

step	vB earliest
EXIT_1 by owner or evictor, with fee input and change	$\approx 1{,}020$ t
update_n bound to the funding output	153 $t+1$
anchor child for update_n	152 $t+1$
$\mathrm{settlement}_n$ (two payouts, anchor)	204 $t+1+\mathsf{csv}$
anchor child for $\mathrm{settlement}_n$	152 $t+1+\mathsf{csv}$
total	$\approx 1{,}680 \approx 146$ blocks
contested rebind to an earlier update output	161 per contest
settlement with h HTLC outputs	$204 + 43h$

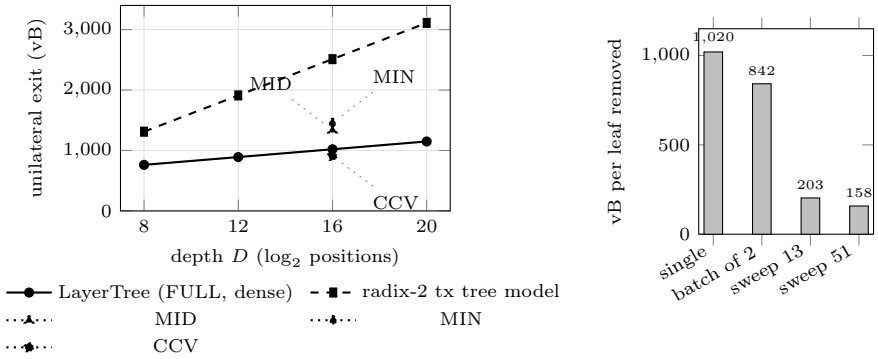


Fig. 4. Left: unilateral exit cost against tree depth, measured for LayerTree and modelled for an unamortised radix-2 transaction tree. Right: per-leaf cost of removal at $D = 16$: a single exit, the average of a two-exit batch, and the cohort sweeps $\mathsf{EVICT_SUBTREE}_4$ (13 leaves) and $\mathsf{EVICT_SUBTREE}_6$ (51 leaves).

The lifetime on-chain footprint of a channel leaf that is force-closed is therefore about 1,680 vB, against roughly 280 vB for a Lightning channel's funding transaction plus force close; cooperative closes, splices and lease renewals of a channel leaf cost one tree transition or nothing. The update transaction cannot be packaged with the exit because a version-3 transaction may not spend an unconfirmed version-2 parent, so G_{exit} in §9 is measured to confirmation.

11 Evaluation

Implementation. A Python tapscript interpreter for both leaf versions models BIP-441 numerics, `OP_TX`, `OP_CHECKSIGFROMSTACK`, the EC opcodes and varops accounting, computes the real BIP-341 signature message and enforces `MINIMALDATA`. Every compiled predicate runs against a reference tree with positive and negative cases per rule (forged leaf, stale root, attacker-chosen m, unconserved amount, underfunded join, early eviction, evictor shortfall, unsigned update, voucher replay, over-claim, stale second path, wrong preimage): 425 FULL, 120 MID, 31 MIN and 309 CCV checks, failing if any script uses an opcode outside the candidate set. A Go implementation on `btcd` and `lnd` generates the same scripts, byte-exact against 39 golden vectors, and its BIP-341 signature hash equals the simulator's, which makes the MIN tier testable end to end.

Exit cost. In the dense profile (Figure 4; Table 6 in Appendix B has the raw sizes) a unilateral exit costs 762 vB at $D = 8$ and 1,149 vB at $D = 20$: a 4,096-fold increase in tree size raises it by 51%, about 32 vB per level. Tier matters more than depth: MID costs 31% and MIN 43% more than FULL. The sparse profile at $D = 32$ with 2^{16} members costs about 1.8 times the dense profile at $D = 16$; its witness grows with $\log_2 N$, not D.

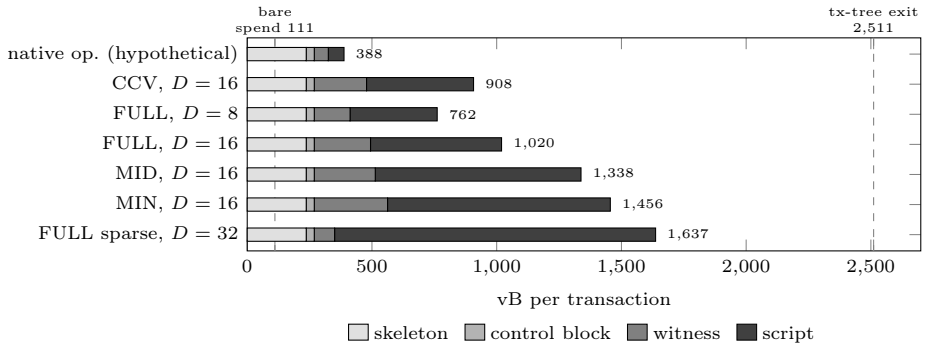


Fig. 5. One unilateral exit by variant, dense unless noted (sparse measured on a six-leaf tree). Skeleton (236.5 vB) and control block (32.25 vB) are common to every row; the script dominates because the levels are unrolled. Dashed: a bare P2TR spend and the transaction-tree model. The native-opcode row measures a hypothetical opcode in the simulator (§18).

Fees and budgets. At 10 sat/vB a $D = 16$ exit costs 10,200 sat: a 1,020,000 sat balance exits for a 1% fee, 102,000 sat for 10% (Appendix B). Varops usage is 5.6 to 7.7% of the BIP-440 budget in the dense profile and 2.5 to 4.7% in the sparse, falling with depth because the budget scales with weight; `0xC0` tiers use one to three of the BIP-342 sigop budget.

12 Costs Against the Alternatives

Joining, updating and leaving a shared output each cost more bytes than the same operation on a bare UTXO. What sharing buys is a different placement of the cost: entry is nearly free in bulk, value moves off-chain for free, a bulk exit costs about a bare spend, and a solo exit costs about ten times one but is a single transaction with no timelock and no counterparty. All figures in this and the following sections use one model: script, witness and control block as measured, plus a 946 WU skeleton (fee input, change, tree output, one P2TR payout) and 172 WU per further payout, vB $= \lceil \mathrm{WU}/4 \rceil$. Figures marked *estimate* are built from measured sizes for scripts not yet generated.

Exit. Figure 5 decomposes exits. The FULL exit at $D = 16$ is 1,020 vB, or 921 vB with the fee taken from the payout. The hypothetical native opcode, at 388 vB, approximates the floor for any design that verifies a proof in script: the proof itself, about 10 vB per level, is still published. A radix-2 transaction tree of the same size, in which the exiter publishes D transactions of about 150 vB and a 111 vB claim (spec 11), needs 2,511 vB; the model flatters it by ignoring the leaf's timelock and the separate fee of each of its 16 transactions. In bulk the cost per leaf depends on who signs. With EXIT_K every leaf carries its own path, so a second exit costs 665 vB; the operator's sweep of expired leases shares the

Table 3. Lifetime on-chain bytes per member who enters in a batch, by how it leaves, and the unilateral exit's latency (a tower watches for stale states; an Ark member must refresh before expiry). Entry is one 43 vB output in a shared transaction, or for LayerTree a share of a 256-slot graft; LayerTree's batch exit is the 51-leaf sweep, which requires a lease; Ark's unilateral exit is the 2,511 vB unroll at $D = 16$; a Lightning force close is a 130 vB commitment and a 110 vB sweep.

	vB per member, leaving			unilateral exit		
	never alone	in a batch	alone	txs	blocks	online
LayerTree dense, $D = 16$	3	161	1,023	1	1	nobody
LayerTree sparse, $D = 32$	1,644	3,108	3,281	1	1	nobody
Lightning channel	197	197	283	2	≈ 145	tower
Ark round tree, $D = 16$	86	86	2,554	17	16+ timelock	member
bare UTXO	154	154	154	1	1	nobody

path and costs 203, 158 and 147 vB per leaf for 13, 51 and 206 leaves, within 1.4 times a bare spend, but only for members who accepted a lease. The owner-side equivalent is the cohort rewrite of §14, estimated at 353 to 177 vB per leaf for 4 to 51 owners. The limit of 25 unconfirmed descendants counts transactions, not leaves.

Entry and moving value. JOIN_h walks only the levels above its slot, so at $D = 16$ a 1,024-member graft costs 657 vB, under 1 vB per member, against 43 vB for one output in an on-chain batch and about 154 vB for a channel opened in its own transaction (§15). An on-chain UPDATE is 1,073 vB and a voucher redemption 1,642 vB, so moving value by rewriting leaves would cost ten times a bare spend per payment. Value moves off-chain instead: in channel leaves as HTLC forwards (§9), by vouchers that the issuer usually settles rather than the holder redeems (§16), or by SPLIT: for 1,000 members moving value 1,000 times each, about 1 kvB for one graft against 111 MvB as bare UTXOs (estimate). UPDATE is maintenance, not payment.

Lifetime and latency. Table 3 adds entry and exit. The dense tree wins outright for a member who never exits alone or who is swept, loses to Lightning and to a bare UTXO for a member who exits alone, and still beats an Ark round tree there by a factor of 2.5; the sparse profile is the most expensive design in every column. A LayerTree exit has no presigned transaction to lose, no counterparty, no timelock and no expiry. It loses on the solo exit, the on-chain per-leaf update, the sparse profile and serialisation through one output.

13 Profiles and Radix

Both profiles are the same Merkle-sum tree. They differ in who decides where a leaf sits, and that choice sets the depth, the proof encoding, which predicates exist and the price of every exit.

Table 4. Radix in the dense FULL tier, measured: a radix-r level generated and run in the simulator against a radix-r reference tree (55 checks), and EXIT_1 for 65,536 slots (262,144 at radix 8). The radix-2 row reproduces the shipped LEVEL.

radix	level script (B)	level siblings (B)	levels	EXIT_1 script (B)	witness (B)	tx (vB)
2	88	41	16	2,101	902	1,020
4	146	123	8	1,861	1,230	1,042
8	215	287	6	1,983	1,968	1,257

One node format. A parent hashes each child's sum, not only their total, and this is a soundness property. A tree hashing $h_L \,\|\, h_R \,\|\, (s_L + s_R)$ admits inflation: two leaves of 100 under a root of sum 100 each have a proof whose sibling claims sum zero, and together the proofs claim 200. Here the two proofs hash the distinct preimages $(A, 100, B, 0)$ and $(A, 0, B, 100)$, so at most one reaches the root; in Lean this is serializeNode_inj with hNode_inj, from which fold_sound gives that a verified leaf carries its true amount.

What the profiles decide. A sparse position is a function of the leaf, so a joiner computes it alone, a key and nonce cannot appear twice, the root depends only on the leaf set, and absence is an inclusion proof of E_0. None of LayerTree's predicates needs these properties: an exit proves inclusion and a join proves its slot is E_h, the same way in both profiles. The dense profile has aligned subtrees instead, on which JOIN_h, $\mathsf{EVICT_SUBTREE}_h$ and every shape of §14 depend.

Where the bytes go. A sparse proof is the smaller witness and the larger script, and the script wins: dense EXIT_1 at $D = 16$ is 2,101 bytes of script and 902 of witness (1,020 vB), sparse at $D = 32$ is 5,147 and 326 on a six-leaf tree (1,637 vB), about 860 bytes of witness and 1,800 vB at 2^{16} members. Both causes follow from hashing the position. Random positions collide, $N^2/2^{D+1}$ expected collisions among N members (about one half at $D = 32$ for $N = 2^{16}$, about 32,000 at $D = 16$), so the sparse tree needs about twice the depth, and every level is unrolled. And a sparse level is 135 bytes against 88, because it derives the direction bit from the position, substitutes E_k for an elided sibling and computes E_{k+1} for the next level. The full-width tree of Taproot Assets [30] ($D = 256$) costs 35,611 bytes of script per exit, about twelve times dense; MID caps sparse depth at 31.

Dense is therefore the default (spec 5.4): 1.6 to 2.7 times cheaper per exit, cohort grafts at one price, sweeps at about 160 vB per leaf, and every batch shape, at the price of an operator who keeps the slot map. The sparse profile is kept, specified and tested, as the measured alternative and for deployments that are key-addressed registries; it still hosts channel leaves and vouchers and still exits in one transaction.

Radix. A radix-r path carries $41(r-1)D/\log_2 r$ sibling bytes, $41D$ at radix 2 and $61.5D$ at radix 4, so radix 2 minimises the witness and a higher radix can only save the fixed part of the unrolled level. Table 4 measures it. Of the radix-4

level's 146 bytes about 62 are hashing and sums shared with radix 2 and the rest sibling and direction handling, so the fixed part a higher radix amortises is about 60 bytes while the sibling part is what it multiplies. Radix 4 saves 240 bytes of script and adds 328 of witness, a net loss of about 3%, and radix 8 is worse. In the sparse profile most siblings are empty and cost one witness byte, so a radix-4 level of about 180 bytes could replace two levels of 135, saving about 360 vB per exit (estimate; no sparse radix-r level has been written). With a native Merkle-sum opcode a level costs only its sibling and radix 2 is optimal.

14 Batching

The path is most of a transition's cost, about 510 vB of a 1,020 vB exit at $D = 16$, so batching is the question of how many leaves can share one. A transition has a fixed part (skeleton, state binding, epilogue, control block; about 470 vB) paid once per transaction, about 32 vB per shared level paid once per path, and about 100 to 120 vB per leaf touched (its 116 bytes, signature, payout and hashing script), which no shape removes (all three are estimates from the measured sizes). Batching therefore helps only leaves under one aligned subtree, which cohort grafts create and the sparse profile cannot.

What exists. EXIT_K chains single-leaf steps, each with its full path, and saves only the skeleton (665 vB for a second exit). $\mathsf{EVICT_SUBTREE}_h$ is the real batch: the witness carries the path above an aligned subtree once and every position of the subtree, the script folds the subtree in post order checking each lease and evictor key, and folds the result and E_h up the shared path (158 vB per leaf for 51 leaves).

The cohort rewrite, $\mathit{REWRITE}_h$. Generalising the sweep gives the general cohort transition. For each position of an aligned subtree the witness carries an old and a new element (leaf or empty marker); the old root, folded from the old elements, must fold with the shared path to (r, S); the new root is folded from the new elements; output 0 is $(S - \sum \text{old} + \sum \text{new}, \mathsf{TreeSPK}(r', m))$, an increase funded by other inputs. The owner of each changed leaf signs, the operator signs once, and a leaf replaced by E_0 must be paid to its exit key from the next unused output, which makes the rewrite a safe mass exit and subsumes the owner multiproof of spec 13.4. In one transaction a cohort can then exit, join, resize, renew, rotate keys and pay between members with the sum conserved, with only members whose balances fall signing: the settlement step of an Ark round without a round, an expiry, or unchanged members online. Exits cost the sweep plus about 64 bytes of signature and 10 of script per leaf, 353, 222 and 177 vB per leaf for 4, 13 and 51 owners, about 1.1 times the operator's sweep; a changed leaf costs about 120 vB and an unchanged one 57 vB (estimates). The Lean model covers it as one Step whose old and new nodes are published subtrees: Steps.sum gives conservation and Steps.sound grounding from the witness, so a cohort that joins this way cannot be opaque.

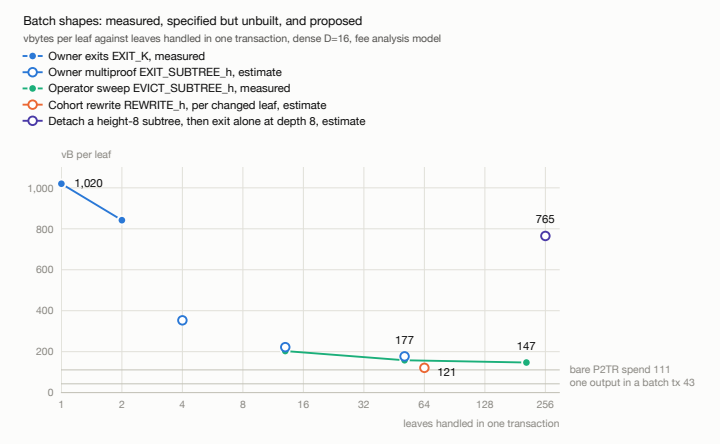


Fig. 6. Batch shapes at $D = 16$ on the model of §12. EXIT_K and the sweep are measured; the owner multiproof (now part of $\mathsf{REWRITE}_h$), the rewrite and the detach are estimates.

Caching the path, DETACH_h. Script cannot read a previous witness, so the upper levels of a cohort's path can only be cached in an output of its own. DETACH_h, signed by the operator, proves an aligned subtree's node (R_h, s_h) against (r, S), leaves E_h in output 0 for $S - s_h$, and creates output 1 as a depth-h tree with root R_h, amount s_h and a taptree m_h fixed at creation; members' paths truncated to the lower h levels are their new proofs. At $D = 16$, $h = 8$ it costs about 815 vB and every later exit 762 vB instead of 1,020 vB (estimates), repaid by the third exit. The larger effect is throughput: the cohort's exits form their own chain of unconfirmed spends, so $\Delta(N)$ applies to it separately, and the operator can shard on demand when, say, channel leaves near their HTLC deadlines. Vouchers bind m and must be reissued under m_h; channel leaves need nothing, as their $\mathsf{chan_id}$ fixes m at creation and a detach leaves every leaf's bytes unchanged. The Lean model needs a two-output step to cover it.

Figure 6 places the shapes. The new ones are variants of the sweep's post-order fold; none lowers the per-leaf floor of about 100 vB for a leaf that leaves.

15 Grafting and Whom It Trusts

JOIN_h replaces an empty node E_h with a node (R_h, s_h) that the operator built off-chain from up to 2^h new leaves, and adds s_h to the tree output. Nothing below height h appears on-chain and the script walks only the $D - h$ levels above the slot, so a larger cohort is a smaller transaction (Table 5). Each height is a tapleaf fixed at creation; the sparse profile batches only by JOINK_K.

What the covenant checks. The predicate requires that the slot held E_h, that output 0 carries $S + s_h$, and the operator's signature. It sees a hash and a sum

Table 5. Entry cost per member at $D = 16$ on the model of §12. JOIN_0 and JOIN_2 are measured, the other dense rows are the measured rows with the same number of levels (JOIN_8 is JOIN_0 at $D = 8$); per member is operator-funded; member-funded adds a 58 vB input per member.

graft	members	tx (vB)	per member	member-funded
JOIN_0	1	980	980	$\approx$ 1,040
JOIN_2	4	915	229	$\approx$ 290
JOIN_4	16	851	53	$\approx$ 110
JOIN_8	256	722	2.8	$\approx$ 61
JOIN_{10}	1,024	657	0.64	$\approx$ 59
sparse JOIN_0, $D = 32$	1	1,644	1,644	$\approx$ 1,700
sparse JOINK_2, $D = 32$	2	2,864	1,432	$\approx$ 1,490

and cannot tell a real cohort from random bytes, which is what makes the graft constant-size and is the source of every trust question. join_sound states the consequences: the slot was empty, output 0 is the old sum plus s_h, and for every tree whose root is the grafted node the new root is the old tree with it inserted. A graft therefore cannot move, shrink or rewrite an existing leaf. A node with no preimage is the opaque slot of $\mathcal{F}_{\mathsf{LT}}$: its s_h, exactly the value the graft added, is frozen, every other leaf stays payable (Theorem 2 holds whatever a sibling is), and the terminal exit is lost.

Who carries what risk. Existing members never sign and take no risk. The operator can fill an empty aligned slot with any node it signs and funds, so an opaque or zero-sum graft freezes only the value it brings and wastes only slots no one else could fill. Assurance is needed only by those who put money in. A member funding with an input checks the unsigned transaction (the slot folds as E_h to the current root, its leaf folds to (R_h, s_h) with sums recomputed, output 0 commits to the new root for $S + s_h$) and signs over all outputs, so its input can only be spent into a root containing its leaf. A member buying in off-chain checks the same path against the confirmed root before paying over Lightning, or pays through a hash-locked voucher (§16). Neither checks anyone else's leaf: leaves whose paths verify sum to at most the root, so a co-member's bad leaf cannot make a good one unpayable.

Funding. In an operator-funded graft the operator supplies s_h from the input the model already counts and members buy their balances off-chain; the operator bears the credit risk of a member who never pays, a risk to it and not to other members. In a member-funded graft each member adds an input, one absent member stalls the cohort (a liveness cost, not a safety one), and the saving against an on-chain batch, 43 vB per output, is gone. A 256-member operator-funded cohort at $D = 16$ is one 722 vB transaction, under 30 sat per member at 10 sat/vB; had the operator grafted a node with no tree behind it, every member's check would fail, nobody would pay, and only the operator's own coins would be frozen.

Designing a cohort. The slot the operator picks is what later batching depends on, and a cohort should share a lease, since one leaf with a NUMS evictor or a

later lease blocks the sweep of its subtree. $\mathsf{REWRITE}_h$ is a graft that publishes its leaves: grounded by construction, at an estimated 60 vB or more per leaf against under 3 for JOIN_8. A graft also settles off-chain credit as exitable leaves (§16).

16 Receiving over Lightning

A user with no on-chain funds receives a Lightning payment and wants a balance it can leave with alone. An Ark client gets an out-of-round output co-signed by the operator that expires with its round, or, below the operator's floor, a ledger credit with no on-chain claim. The tree takes the same payment with less trust.

Cumulative hash-locked vouchers. A CLAIM^h voucher (Appendix B.2) is the operator's signature over a source leaf it owns, a claim leaf for the recipient and a payment hash; whoever holds it and the preimage can redeem it alone. The recipient generates the preimage and issues the invoice, the payer's HTLC reaches the operator, the operator issues a voucher under that hash, and the recipient reveals the preimage so the operator can settle. Nothing reaches the chain. Since a redemption advances the source leaf's seq and voids every other voucher against it, repeated receives are one *cumulative* voucher that the operator reissues for the running total; the recipient keeps the latest, and needs no contest window because only it can redeem and it never prefers a smaller state.

Settlement. The voucher is insurance. Normally the operator puts the claim leaf into the next cohort graft at about one vbyte per recipient (§15) and retires the voucher by advancing the credit leaf's seq: one UPDATE today (1,073 vB), about 120 to 240 vB per recipient with $\mathsf{REWRITE}_h$ (estimate). Unilaterally the recipient redeems one CLAIM^h: 1,652 vB at $D = 16$, about 16,500 sat at 10 sat/vB, no timelock, against about 2,511 vB and a timelock to unroll an Ark round tree. The redemption proves two positions; if the credit leaf's sibling slot is reserved for the recipient's claim leaf, a sibling-pair CLAIM shares fifteen of sixteen levels and costs about 1,130 vB (estimate, not built). Per receive nothing reaches the chain, there is no floor beyond dust at redemption, nothing expires, and the recipient need not be online.

What the recipient trusts. The covenant stops the operator from redeeming more than the source holds or paying anyone but the recipient. It does not stop over-issuance or voiding (advancing seq before the recipient redeems). A credit leaf dedicated to one recipient removes over-issuance, since every voucher against it goes to that recipient, who can watch its amount. Of the operator's ways to void, UPDATE^x (the exit key, which is the recipient's, must sign), exact payouts and a NUMS evictor close three; the fourth, a voucher the operator issues itself and redeems with plain CLAIM, is closed only by a CLAIM^x variant requiring a signature by the source leaf's exit key. With the three variants in the taptree a credit leaf can only grow or pay its recipient, whose exposure is the operator's liveness. CLAIM^x is proposed, not specified; the Lean model is

unaffected, since Claim carries an opaque authorisation. Where one operator leaf backs many recipients, over-issuance is equivocation with a compact proof, which a slashable bond can deter; voiding has no compact proof, so no bond reaches it.

The credit is one-way: if the recipient could also spend, an older, larger voucher would be a stale state needing a window to supersede, as in every bidirectional channel. For spending, the recipient redeems into a channel leaf (§9).

17 Related Work

CoinPool [34] is the closest prior design: a shared UTXO with per-user balances and unilateral withdrawal, realised with pre-signed or covenant-committed transactions, so exit and update remain transaction trees. Channel factories [5] need all N members to sign every change and publish a tree to exit. Ark [20] batches users into rounds whose virtual outputs are leaves of a pre-signed tree; covenant Ark [21] removes the signing round with CTV or CCV templates but keeps the tree, its unrolled exit and its expiry. Timeout trees [22] and SuperScalar [45] combine covenant trees with Decker-Wattenhofer factories and laddered expiries; MATT-style trees [17,18] replace signatures by templates. Statechains [39] transfer a whole UTXO off-chain but do not divide it; `OP_VAULT` [28] is an introspection covenant that, like ours, recreates its output under a modified commitment. In LayerTree the tree is data rather than transactions: exit is one transaction of $O(D)$ bytes, the interior needs no pre-signing, the operator can evict, and the output never expires (Table 7, Appendix B). None states its guarantees as a functionality; the closest treatments are the ledger model of Badertscher et al. [2] and the channel functionalities of [1,14], which we build on.

The Merkle-sum sparse Merkle tree and its compressed proofs come from Taproot Assets [30]; here the tree is truncated and verified in script. The data-tweaked internal key follows `OP_CHECKCONTRACTVERIFY` [18]. The channel state machine is eltoo [11], with `ANYPREVOUT` [12] emulated by collated introspection and `OP_CHECKSIGFROMSTACK`. Liquid's tapscript [4] has `OP_CAT`, `OP_CHECKSIGFROMSTACK`, per-field introspection, 64-bit arithmetic and point-tweak verification, so the MID tier is close to expressible there today; Simplicity [29] would express the predicates directly. We know of no prior use of a generator-key signature with $s = e + 1$ to make a script's sighash reconstruction binding (§5).

18 Discussion and Limitations

The functionality of §3 is deliberately modest: it promises conservation and the payability of every leaf that is not opaque, and nothing about being admitted, about grafted subtrees having preimages, or about the tree having been created correctly; the members' checks are mandatory and the Lean development shows they are load-bearing. Stating the design this way made visible that opacity is a state of the functionality rather than a failure of the protocol, and that $\Delta(N)$ is a consequence of the single-UTXO design rather than a tunable: trees should

be sized by exit throughput and sharded. The design is hostage to proposals (a tapleaf version with restored string opcodes or at least `OP_CAT`, `OP_TX` in `0xC0`, the EC opcodes' drafts, or BIP-443 for CCV); every selector and opcode number is a named constant, and a Liquid prototype of the MID tier is the obvious next experiment. As a thought experiment (simulator only, never a proposal), a hypothetical native Merkle-sum update opcode that folds the path inside consensus takes EXIT_1 to 252 B of script at every depth and profile, sparse and dense converging, at the price of baking the tree format into consensus (design notes).

19 Conclusion

A shared output can be specified by what it lets its members do: join, leave alone, be evicted when a lease expires, be swept as a cohort, with the amount it holds always the sum of its leaves. LayerTree realises that functionality with a tree that lives off-chain as data rather than transactions: a one-transaction exit of about a kilobyte, eviction and cohort sweeps, no pre-signing, no expiry, only opcodes already on the table. Its guarantees are a simulation argument over mechanically checked lemmas, its costs are measured, and its price is that all activity funnels through one UTXO, which bounds how fast a tree can be dissolved. Future work: a verified compiler from Table 1 to bytecode, closing the gap Theorem 5 leaves; a full proof of Theorem 6; a P model [13] of the off-chain protocol; and completing the MIN tier.

References

1. Aumayr, L., Ersoy, O., Erwig, A., Faust, S., Hostáková, K., Maffei, M., Moreno-Sanchez, P., Riahi, S.: Generalized channels from limited blockchain scripts and adaptor signatures. In: ASIACRYPT 2021. LNCS, vol. 13091, pp. 635–664. Springer (2021)
2. Badertscher, C., Maurer, U., Tschudi, D., Zikas, V.: Bitcoin as a transaction ledger: A composable treatment. In: CRYPTO 2017. LNCS, vol. 10401, pp. 324–356. Springer (2017)
3. Black, B., Rubin, J.: BIP-348: CHECKSIGFROMSTACK. `https://github.com/bitcoin/bips/blob/master/bip-0348.md` (2024)
4. Blockstream: Elements tapscript opcodes: Introspection, streaming SHA256, 64-bit arithmetic, ECMULSCALARVERIFY, TWEAKVERIFY. `https://github.com/ElementsProject/elements/blob/master/doc/tapscript_opcodes.md` (2021)
5. Burchert, C., Decker, C., Wattenhofer, R.: Scalable funding of bitcoin micropayment channel networks. In: Royal Society Open Science. vol. 5 (2018)
6. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: FOCS 2001. pp. 136–145. IEEE (2001)
7. Canetti, R.: Universally composable signature, certification, and authentication. In: CSFW 2004. pp. 219–233. IEEE (2004)
8. Canetti, R., Dodis, Y., Pass, R., Walfish, S.: Universally composable security with global setup. In: TCC 2007. LNCS, vol. 4392, pp. 61–85. Springer (2007)

9. Code, R., et al.: BIP-449: OP_TWEAKADD. https://github.com/bitcoin/bips/ (2025)
10. Dahlberg, R., Pulls, T., Peeters, R.: Efficient sparse Merkle trees: Caching strategies and secure (non-)membership proofs. In: NordSec 2016. LNCS, vol. 10014, pp. 199–215. Springer (2016)
11. Decker, C., Russell, R., Osuntokun, O.: eltoo: A simple layer2 protocol for bitcoin. https://blockstream.com/eltoo.pdf (2018)
12. Decker, C., Towns, A.: BIP-118: SIGHASH_ANYPREVOUT for taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0118.mediawiki (2017)
13. Desai, A., Gupta, V., Jackson, E., Qadeer, S., Rajamani, S., Zufferey, D.: P: Safe asynchronous event-driven programming. In: PLDI 2013. pp. 321–332. ACM (2013)
14. Dziembowski, S., Eckey, L., Faust, S., Malinowski, D.: PERUN: Virtual payment hubs over cryptocurrencies. In: IEEE S&P 2019. pp. 106–123. IEEE (2019)
15. Harding, D.A., Todd, P.: BIP-125: Opt-in full replace-by-fee signaling. https://github.com/bitcoin/bips/blob/master/bip-0125.mediawiki (2015)
16. Heilman, E., Sabouri, A.: BIP-347: OP_CAT in tapscript. https://github.com/bitcoin/bips/blob/master/bip-0347.mediawiki (2023)
17. Ingala, S.: Merkleize all the things. bitcoin-dev mailing list, https://merkle.fun (2022)
18. Ingala, S.: BIP-443: OP_CHECKCONTRACTVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0443.md (2025)
19. Katz, J., Maurer, U., Tackmann, B., Zikas, V.: Universally composable synchronous computation. In: TCC 2013. LNCS, vol. 7785, pp. 477–498. Springer (2013)
20. Keceli, B.: Ark: An off-chain payment protocol. https://arkdev.info/ (2023)
21. Labs, A.: Covenant-based Ark (clArk): Rounds without presigning. https://arkdev.info/docs/learn/concepts (2024)
22. Law, J.: Scaling lightning with simple covenants. bitcoin-dev mailing list, https://github.com/JohnLaw2/ln-scaling-covenants (2023)
23. Merkle, R.C.: A digital signature based on a conventional encryption function. In: CRYPTO 1987. LNCS, vol. 293, pp. 369–378. Springer (1988)
24. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: CADE 2021. LNCS, vol. 12699, pp. 625–635. Springer (2021)
25. Myers, R., et al.: LN-Symmetry (eltoo) implementation notes. https://github.com/instagibbs/bolts/blob/eltoo_draft/XX-eltoo-transactions.md (2023)
26. Nick, J., Ruffing, T., Jin, E.: BIP-327: MuSig2 for BIP340-compatible multi-signatures. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki (2022)
27. Nick, J., Ruffing, T., Seurin, Y.: MuSig2: Simple two-round Schnorr multi-signatures. In: CRYPTO 2021. LNCS, vol. 12825, pp. 189–221. Springer (2021)
28. O'Beirne, J., Sanders, G., Towns, A.: BIP-345: OP_VAULT. https://github.com/bitcoin/bips/blob/master/bip-0345.mediawiki (2023)
29. O'Connor, R.: Simplicity: A new language for blockchains. PLAS 2017, https://blockstream.com/simplicity.pdf (2017)
30. Osuntokun, O.: Taproot assets: Merkle sum sparse merkle trees. https://github.com/lightninglabs/taproot-assets/blob/main/docs/bip-tap-ms-smt.mediawiki (2022)
31. Osuntokun, O.: Elliptic curve operations for bitcoin script. Draft BIP, https://github.com/bitcoin/bips/pull/1945 (2025)
32. Osuntokun, O.: LayerTree: Specification, reference implementation and formal model. https://github.com/roasbeef/layertree (2026)

33. Poon, J., Dryja, T.: The bitcoin lightning network: Scalable off-chain instant payments. https://lightning.network/lightning-network-paper.pdf (2016)
34. Riard, A., Naumenko, G.: CoinPool: Efficient off-chain payment pools for Bitcoin. https://coinpool.dev/v0.1.pdf (2022)
35. Rubin, J.: BIP-119: CHECKTEMPLATEVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki (2020)
36. Russell, R.: OP_TX: Transaction introspection for tapscript. Draft BIP, https://github.com/rustyrussell/bips/pull/1 (2025)
37. Russell, R., Towns, A.: BIP-440: Varops budget for script runtime constraint. https://github.com/bitcoin/bips/blob/master/bip-0440.md (2025)
38. Russell, R., Towns, A.: BIP-441: Restoration of disabled script opcodes (tapleaf 0xC2). https://github.com/bitcoin/bips/blob/master/bip-0441.md (2025)
39. Somsen, R.: Statechains: Off-chain transfer of UTXO ownership. In: Scaling Bitcoin 2018 (2018)
40. Wuille, P., Daftuar, S., Zhao, G.: Cluster mempool. Delving Bitcoin, https://delvingbitcoin.org/t/an-overview-of-the-cluster-mempool-proposal/393 (2023)
41. Wuille, P., Nick, J., Ruffing, T.: BIP-340: Schnorr signatures for secp256k1. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki (2020)
42. Wuille, P., Nick, J., Towns, A.: BIP-341: Taproot: SegWit version 1 spending rules. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki (2020)
43. Wuille, P., Nick, J., Towns, A.: BIP-342: Validation of taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki (2020)
44. Zhao, G.: BIP-431: Topology restrictions for pinning. https://github.com/bitcoin/bips/blob/master/bip-0431.mediawiki (2024)
45. ZmnSCPxj: SuperScalar: Laddered timeout-tree-structured Decker-Wattenhofer factories. Delving Bitcoin, https://delvingbitcoin.org/t/superscalar-laddered-timeout-tree-structured-decker-wattenhofer-factories/1143 (2024)

A Script Listings

Figure 7 states the step every transition is built from; the listings below are taken from the reference generators and are what the size figures of §11 measure. Stack comments show the main stack left to right with the top on the right, and the alt stack after a bar.

One level of the fold (FULL tier, 88 bytes). LEVEL advances both accumulators, the old node (oh, os) and the new node (nh, ns), by one level using the same direction bit and sibling $sc = h \,\|\, \mathsf{enc}_{64}(s)$ from the witness. T is the 64-byte tag prefix of H_{node}; PAD8 is $\langle 2^{64} \rangle$ OP_ADD, which yields enc_{64} of the sum by Lemma 1.

```
OP_6 OP_ROLL OP_6 OP_ROLL            # T oh os nh ns dir sc
OP_SIZE 41 OP_EQUALVERIFY            #   sc is exactly hash || enc64(sum)
OP_DUP 32 OP_8 OP_SUBSTR             # ... dir sc ss8   (sibling sum, raw le64)
OP_DUP OP_TOALTSTACK                 #                          | ss8
OP_3 OP_ROLL OP_TUCK OP_ADD          # T oh os nh dir sc ns ns'
OP_TOALTSTACK PAD8                   # T oh os nh dir sc ns9    | ss8 ns'
OP_3 OP_ROLL OP_SWAP OP_CAT          # T oh os dir sc nc        (nc = nh || enc64(ns))
OP_OVER OP_SWAP OP_3 OP_PICK         # T oh os dir sc sc nc dir
OP_NOTIF OP_SWAP OP_ENDIF OP_CAT     # T oh os dir sc pre_n     (dir ? sc||nc : nc||sc)
```

$\mathsf{Step}(\pi, n_{\text{old}}, n_{\text{new}}, (r, S))$

1 : $(r_0, S_0) \leftarrow \mathsf{fold}(n_{\text{old}}, \pi)$ // $D - h$ unrolled levels
2 : **assert** $r_0 = r \wedge S_0 = S$ // CHECK_OLD: S is input 0's amount
3 : $(r', S') \leftarrow \mathsf{fold}(n_{\text{new}}, \pi)$ // same siblings, same sums
4 : **return** (r', S')

$\mathsf{Final}((r', S'), m)$

1 : **if** $(r', S') = E_D$ **then return** // terminal: output 0 free
2 : $(a_0, \mathsf{spk}_0) \leftarrow$ OP_TX(output 0)
3 : **assert** $\mathsf{spk}_0 = \mathsf{TreeSPK}(r', m) \wedge a_0 = S'$

Fig. 7. The Merkle-sum step shared by all transitions, and the output rule. A transition is a sequence of steps, each proving its old node against the root the previous step produced (CHECK_PREV), followed by Final. Authorisation and payout checks are specific to each transition.

```
OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 # T oh os dir sc nh'
OP_TOALTSTACK                         #                          | ss8 ns' nh'
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
                                      # T oh os dir sc nh' ns' ss8
OP_5 OP_ROLL OP_TUCK OP_ADD           # T oh dir sc nh' ns' os os'
OP_TOALTSTACK PAD8                    # T oh dir sc nh' ns' os9  | os'
OP_5 OP_ROLL OP_SWAP OP_CAT           # T dir sc nh' ns' oc      (oc = oh || enc64(os))
OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL     # T nh' ns' sc oc dir
OP_NOTIF OP_SWAP OP_ENDIF OP_CAT      # T nh' ns' pre_o
OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256 # T nh' ns' oh'
OP_FROMALTSTACK OP_2SWAP              # T oh' os' nh' ns'
```

Checking the old root and writing the new one. After D levels, CHECK_OLD compares the old accumulator with the committed state: the sum against input 0's amount and the hash against the witness root r parked on the alt stack (itself bound to the input scriptPubKey by the prologue). FINAL constrains output 0 unless the new root is the empty tree.

```
# CHECK_OLD:  [T oh os nh ns] | ... r  ->  [T nh ns]
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

# FINAL:  [T nh ns] | m  ->  [1]
OP_ROT OP_DROP                                  # nh ns
OP_OVER <E_D.hash> OP_EQUAL
OP_IF   OP_2DROP OP_FROMALTSTACK OP_DROP OP_1   # terminal: output 0 free
OP_ELSE OP_SWAP OP_FROMALTSTACK TREE_SPK        # ns spk'
        OP_0 <SEL_OUT_SINGLE> OP_TX             # ns spk' amt0 spk0
        OP_ROT OP_EQUALVERIFY OP_NUMEQUAL       # 1
OP_ENDIF
```

The MID gadget. Under 0xC0 a computed 4-byte number x is widened to 8 bytes by two witness elements z, w: the script checks $\mathsf{size}(x) + \mathsf{size}(z) = 4$ and $z \,\|\, w = 00000000$ and uses $x \,\|\, z \,\|\, w$ as $\mathsf{le}_{64}(x)$. Two such gadgets per level and the per-field size checks account for almost all of MID's premium over FULL.

The MIN message. The MIN exit assembles the BIP-341 signature message for input 0 with hash type `SIGHASH_DEFAULT` and checks it as described in §5. Fields marked *derived* are computed by the script from values it has verified; fields marked *raw* are witness elements the script does not constrain but must be presented consistently, since the whole message is hashed and compared with the real one through `OP_CHECKSIG`.

field	source	note
epoch, hash type	constant `00 00`	`SIGHASH_DEFAULT`
version, lock time	raw	free
sha_prevouts	raw	free
sha_amounts	derived: $\mathsf{SHA256}(\mathsf{le}_{64}(S) \,\|\|\, amt_1)$	S verified by CHECK_OLD; amt_1 raw; two inputs exactly
sha_scriptpubkeys	derived: $\mathsf{SHA256}(22 \,\|\|\, \mathsf{TreeSPK}(r, m) \,\|\|\, spk_1)$	TreeSPK recomputed; spk_1 raw
sha_sequences	raw	the field the spender grinds
sha_outputs	derived: $\mathsf{SHA256}(out_0 \,\|\|\, out_1 \,\|\|\, extra)$	$out_0 = \mathsf{le}_{64}(S') \,\|\|\, 22 \,\|\|\, \mathsf{TreeSPK}(r', m)$, out_1 the payout; $extra$ raw
spend type, input index	constant `02 00000000`	script path, no annex, input 0
tapleaf hash	raw	the leaf being executed
key version, codesep	constant `00 ffffffff`	

The message is hashed with the TapSighash tag and the challenge e is computed from $G_x \,\|\, G_x \,\|\, \mathsf{msg}$ under the challenge tag. The script verifies $e_{[0,31)} \,\|\, \mathtt{00} = e$ against the witness-supplied $e_{[0,31)}$ and finally executes

$$\langle G_x \,\|\, e_{[0,31)} \,\|\, \mathtt{01} \rangle \;\; \langle G_x \rangle \;\; \mathtt{OP_CHECKSIG}.$$

B Measurements, Vouchers and Fee Tables

B.1 Measured Sizes

Table 6 gives the script, witness and transaction sizes behind Figure 4 and Table 2.

Table 6. Measured EXIT_1 cost (FULL). WU include a 129-byte control block for the eight-leaf reference script set (each doubling of the set adds 8 vB); the transaction adds a fee input, change and overhead (≈ 950 WU). Sparse witness is $326 + 41 \log_2 N$ bytes, tabulated at $N = 2^{16}$. Last column: unamortised radix-2 transaction tree of 2^D leaves, $150D + 111$ vB; sparse D is address width.

profile	D	script (B)	witness (B)	total (WU)	tx (vB)	radix-2 tx tree (vB)
dense	8	1,397	574	2,100	762	1,311
dense	12	1,749	738	2,616	891	1,911
dense	16	2,101	902	3,132	1,020	2,511
dense	20	2,453	1,066	3,648	1,149	3,111
sparse	32	5,147	$326 + 41 \log_2 N$	$\approx 6{,}260$	$\approx 1{,}800$	n/a
sparse	64	9,499	$326 + 41 \log_2 N$	$\approx 10{,}610$	$\approx 2{,}890$	n/a
MID dense	16	3,299	978	4,406	1,338	
MIN dense	16	3,572	1,175	4,876	1,456	

B.2 Vouchers (CLAIM)

A voucher is a BIP-340 signature by the source leaf's owner over

$$H_{\mathsf{voucher}}(m \,\|\, \ell_{\mathrm{src}} \,\|\, \ell_{\mathrm{claim}}),$$

verified with `OP_CHECKSIGFROMSTACK`. CLAIM is two steps: the source leaf's amount is reduced by the claim amount and its seq advanced by one (or the position emptied if drained), then the claim leaf is placed at an empty position. The total is unchanged. Replay is impossible because the first step changes the source leaf, so the signed message never again matches a leaf in the tree; seq prevents a balance that later returns to an earlier value from re-enabling an old voucher. Because a voucher lets an owner move value with a lone signature, leaves whose balance is not the owner's alone set seq to the tier's SEQ_FINAL (§9), whose successor does not fit in four bytes, so the CLAIM script rejects them. Vouchers are the mechanism for operator-to-member push payments without the operator's later participation.

Hash-locked vouchers (CLAIM^h). Adding a payment hash to the signed message, under its own tag, and requiring the preimage at redemption turns a voucher into an HTLC and makes it composable with a channel-leaf HTLC or a Lightning invoice carrying the same hash. The script does not compare a witness-supplied hash against the preimage: it computes SHA256(preimage) and feeds it into the signed message, so a wrong preimage fails as an invalid signature. That costs one hash opcode, nine bytes of script in both tiers that realise CLAIM, and no opcode beyond those CLAIM already needs. The separate tag is load-bearing: reusing H_{voucher} for both forms would let a redeemer strip the hash lock.

The variant adds ordering, not security. It cannot create an on-chain deadline, because script can require that a spend not happen before a height but not that it happen before one, and the value backing a voucher never leaves the source leaf, so there is nothing to refund; deadlines live in the other leg of the swap, and an issuer retires an unredeemed voucher by advancing the source leaf's seq. Redemption publishes the preimage, which is what settles the other leg and also what obliges a redeemer to secure its own leg first.

B.3 Comparison Table

B.4 Fee Tables

Fees for the measured sizes at fee rates from 1 to 250 sat/vB, FULL tier. The exogenous-fee skeleton is 946 WU for one payout plus 172 WU per additional payout; vB = $\lceil \mathrm{WU}/4 \rceil$.

Table 7. Comparison with shared-output constructions.

	LayerTree	CoinPool	Ark (covenant)	Timeout tree	Factory
Unilateral exit	1 tx, $O(D)$ B	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs
Evict one leaf	1 tx after lease	no	tree expires	tree expires	no
Interactive signing	none	all	none	none or all	all
Shared output expiry	none	none	per round	per tree	none
Batch add	1 tx, any subtree	re-sign all	per round	per tree	re-sign all
Opcodes	GSR, `OP_TX`, CSFS, EC	CTV or APO	CTV / CCV	CTV	none

case	vB	1	10	25	50	250 sat/vB
dense EXIT_1, $D = 8$	762	762	7,620	19,050	38,100	190,500
dense EXIT_1, $D = 16$	1,020	1,020	10,200	25,500	51,000	255,000
dense EXIT_1, $D = 20$	1,149	1,149	11,490	28,725	57,450	287,250
sparse EXIT_1, $D = 32$, $N = 6$	1,637	1,637	16,370	40,925	81,850	409,250
MID dense EXIT_1, $D = 16$	1,338	1,338	13,380	33,450	66,900	334,500
MIN dense EXIT_1, $D = 16$	1,456	1,456	14,560	36,400	72,800	364,000
second exit in EXIT_2, marginal	665	665	6,650	16,625	33,250	166,250
$\mathsf{EVICT_SUBTREE}_4$, per leaf (13)	203	203	2,029	5,073	10,146	50,731
$\mathsf{EVICT_SUBTREE}_6$, per leaf (51)	158	158	1,580	3,950	7,899	39,495
plain P2TR to P2TR	111	111	1,110	2,775	5,550	27,750

Break-even balances for the dense $D = 16$ exit, i.e. the leaf amount at which the fee is 1%, 5% or 10% of the balance:

sat/vB	fee (sat)	balance at 1%	at 5%	at 10%
2	2,040	204,000	40,800	20,400
10	10,200	1,020,000	204,000	102,000
50	51,000	5,100,000	1,020,000	510,000
250	255,000	25,500,000	5,100,000	2,550,000

Operators should size cohorts and minimum leaf amounts so that a sweep stays below about 10% of the smallest leaf at the fee-rate tail they intend to survive; the FULL-tier fee-from-payout option saves a constant 99 vB and is unavailable to channel leaves.

C Complete Script Listings

This appendix reproduces every script the reference generators emit, for the smallest depths that exercise every construct: dense $D = 4$ and sparse $D = 8$. Tapscript has no subroutines, so a compiled script is one flat opcode stream; the generators, however, build every script from a small set of helpers, and the listings below are those streams *collapsed back* into named macro calls, one per line, with the remaining raw opcodes wrapped between them. Each tier's macros are defined once in §C.1 to §C.8, themselves collapsed against the other macros. The collapse is mechanical (`tools/gen_structured.py`) and verified: expanding a structured listing reproduces the compiled bytes exactly, so these listings and

the flat ones in `spec/scripts/` describe the same scripts, and the byte count in each heading is the compiled size. A run of n identical calls is written `NAME` xn. Angle brackets denote pushes of named constants: `<TAG_NODE>` is the 64-byte tagged-hash prefix, `<EMPTY_k>` the hash of E_k, `<ENC0>` the constant 2^{64} whose addition implements PAD8, `<NUMS>` the BIP-341 nothing-up-my-sleeve point, and `<SEL_*>` the `OP_TX` selectors of §4.2. Bare integers are minimal number pushes. Sizes at $D = 16$ and $D = 32$ are in Table 6; each further level adds one LEVEL (88 bytes dense, 135 sparse, 156 MID).

C.1 FULL Tier Macros

```
# Macro definitions, FULL tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(VOUCHER)
<TAG_VOUCHER> OP_SWAP OP_CAT OP_SHA256

## TAGGED(POS)
<TAG_POS> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## PAD8   ([n] -> [enc64(n)]: minimal encoding of n + 2^64 is le64(n)||01)
<ENC0> OP_ADD

## LE64   ([n] -> [le64(n)])
PAD8
OP_8 OP_LEFT

## PAD4   ([n] -> [le32(n)])
<ZERO4> OP_CAT OP_4 OP_LEFT

## TREE_SPK   ([r m] -> [TREE_SPK(r, m)] with EC ops)
OP_SWAP
TAGGED(STATE)
OP_0 OP_SWAP OP_EC_POINT_MUL <NUMS> OP_EC_POINT_ADD OP_EC_POINT_X_COORD
  OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_0 OP_SWAP OP_EC_POINT_MUL OP_SWAP OP_2 OP_SWAP OP_CAT OP_EC_POINT_ADD
  OP_EC_POINT_X_COORD <0x5120> OP_SWAP OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## LEAF_ACC   ([leaf] -> [H_leaf(leaf) amount])
OP_DUP OP_8 OP_LEFT OP_SWAP
TAGGED(LEAF)
OP_SWAP
```

```
## LEAF_POS   ([leaf] -> [leaf H_pos(owner_key||nonce)])
OP_DUP 40 32 OP_SUBSTR OP_OVER 108 OP_4 OP_SUBSTR OP_CAT
TAGGED(POS)

## PROLOGUE   (input is #0; (r, m) match the input scriptPubKey; alt <- m r)
<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK

## LEVEL   (dense level: dir sc T oh os nh ns -> T oh' os' nh' ns')
OP_6 OP_ROLL OP_6 OP_ROLL OP_SIZE 41 OP_EQUALVERIFY OP_DUP 32 OP_8 OP_SUBSTR
  OP_DUP OP_TOALTSTACK OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## LEVEL_MIN   (dense level without splice opcodes: sibling as two elements)
OP_7 OP_ROLL OP_7 OP_ROLL OP_7 OP_ROLL OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER
  OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_DUP OP_TOALTSTACK OP_SWAP OP_OVER
PAD8
OP_CAT OP_SWAP OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## SPARSE_SETUP   ([pos_hash] -> [] ; alt <- pos E_0)
OP_TOALTSTACK <EMPTY_0> OP_TOALTSTACK

## SPARSE_TEARDOWN   (drop pos and E_k from the alt stack)
OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK OP_DROP

## CHECK_OLD   (os == input 0 amount, oh == r (from alt))
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY OP_FROMALTSTACK
  OP_EQUALVERIFY

## CHECK_OLD_S   (CHECK_OLD with the sparse teardown)
OP_2SWAP <SEL_IN_AMOUNT> OP_TX OP_NUMEQUALVERIFY
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV   (os == ns_prev, oh == nh_prev (from alt); chained steps)
OP_2SWAP OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV_S   (CHECK_PREV with the sparse teardown)
OP_2SWAP
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## LEAF_STEP   (cohort sweep: one leaf or empty -> its node, paying the next output)
OP_SIZE OP_ONOTEQUAL OP_IF OP_SIZE 116 OP_EQUALVERIFY OP_DUP 72 32 OP_SUBSTR
  <OPERATOR_KEY> OP_EQUALVERIFY OP_DUP 104 OP_4 OP_SUBSTR
  OP_CHECKLOCKTIMEVERIFY OP_DROP OP_FROMALTSTACK OP_DUP OP_1ADD OP_TOALTSTACK
  <SEL_OUT_SINGLE> OP_TX <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT
  OP_EQUALVERIFY OP_OVER OP_8 OP_LEFT OP_NUMEQUALVERIFY
LEAF_ACC
OP_ELSE OP_DROP <EMPTY_0> OP_0 OP_ENDIF
```

```
## LEVEL_S(k)   [shown for k = 0; <EMPTY_k> and the bit index vary with k]   (sparse level
     k: sc T oh os nh ns | pos E_k -> T oh' os' nh' ns' | pos E_k+1)
OP_5 OP_ROLL OP_FROMALTSTACK OP_SWAP OP_SIZE OP_ONOTEQUAL OP_NOTIF OP_DROP
  OP_DUP <ENC0> OP_CAT OP_ENDIF OP_SWAP <ENC0> OP_CAT OP_DUP OP_CAT OP_6
  OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_DUP OP_TOALTSTACK
  OP_SWAP OP_TOALTSTACK OP_0 OP_DOWNSHIFT OP_1 OP_AND OP_ONOTEQUAL OP_SWAP
  OP_SIZE 41 OP_EQUALVERIFY OP_DUP 32 OP_8 OP_SUBSTR OP_DUP OP_TOALTSTACK
  OP_3 OP_ROLL OP_TUCK OP_ADD OP_TOALTSTACK
PAD8
OP_3 OP_ROLL OP_SWAP OP_CAT OP_OVER OP_SWAP OP_3 OP_PICK OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_5 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_5 OP_ROLL OP_TUCK OP_ADD
  OP_TOALTSTACK
PAD8
OP_5 OP_ROLL OP_SWAP OP_CAT OP_3 OP_ROLL OP_SWAP OP_4 OP_ROLL OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_3 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK OP_2SWAP

## FINAL   [shown for D = 4; <EMPTY_D> is the empty root of the tree's depth]   (output 0 is
     (ns, TREE_SPK(nh, m)) unless nh == E_D)
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_FROMALTSTACK
  OP_DROP OP_1 OP_ELSE OP_SWAP OP_FROMALTSTACK
TREE_SPK
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_NUMEQUAL OP_ENDIF

## HASH_STEP(t)   [shown for t = 3; the OP_PICK depth is t-3]   (cohort sweep: two child
     nodes -> parent node)
OP_2 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK
PAD8
OP_CAT OP_SWAP
PAD8
OP_ROT OP_SWAP OP_CAT OP_SWAP OP_CAT OP_0 OP_PICK OP_SWAP OP_CAT OP_SHA256
  OP_FROMALTSTACK
```

C.2 FULL Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (dense, D=4): 1045 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_OLD
FINAL

# EXIT_2 (dense, D=4): 1635 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_OLD
```

```
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_DUP OP_TOALTSTACK OP_IF
  OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY OP_DROP OP_OVER 72 32
  OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF OP_CHECKSIGVERIFY
  OP_SIZE 116 OP_EQUALVERIFY OP_2 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_3
  OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8 OP_LEFT
  OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL  x4
CHECK_PREV
FINAL

# JOIN_0 (dense, D=4): 961 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_OLD
FINAL

# JOIN_2 (dense, D=4): 785 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_2> OP_0 OP_2SWAP
LEVEL  x2
CHECK_OLD
FINAL

# JOINK_2 (dense, D=4): 1433 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE OP_8 OP_EQUALVERIFY OP_OVER OP_SIZE
  32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_SIZE OP_8 OP_EQUALVERIFY
  OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP <TAG_NODE> OP_ROT OP_ROT
  <EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL

# UPDATE (dense, D=4): 1081 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116
  OP_EQUALVERIFY <TAG_NODE> OP_ROT OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL  x4
CHECK_OLD
FINAL

# UPDATE_X (exit-key consent, dense, D=4): 1088 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_4 OP_PICK OP_8 32 OP_SUBSTR OP_CHECKSIGVERIFY <OPERATOR_KEY>
```

```
  OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGVERIFY OP_SIZE
  116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY <TAG_NODE> OP_ROT
  OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL  x4
CHECK_OLD
FINAL

# CLAIM (dense, D=4): 1690 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_DUP OP_TOALTSTACK OP_SWAP OP_TOALTSTACK
  OP_3 OP_PICK OP_3 OP_PICK OP_CAT OP_CAT
TAGGED(VOUCHER)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY OP_SIZE 116
  OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_OVER OP_8
  OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK <TAG_NODE> OP_ROT
  OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ENDIF
LEVEL  x4
CHECK_OLD
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL  x4
CHECK_PREV
FINAL
```

C.3 FULL Tier, Cohort Sweep ($D = 8$, $h = 2$)

```
# EVICT_SUBTREE_2 (dense, D=8): cohort sweep of a 4-position subtree: 2005 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_1 OP_TOALTSTACK <TAG_NODE> OP_1 OP_ROLL
LEAF_STEP
OP_3 OP_ROLL
LEAF_STEP
HASH_STEP(4)
OP_3 OP_ROLL
LEAF_STEP
OP_5 OP_ROLL
LEAF_STEP
HASH_STEP(6)
HASH_STEP(4)
OP_FROMALTSTACK OP_DROP <EMPTY_2> OP_0
LEVEL  x6
CHECK_OLD
FINAL
```

C.4 FULL Tier, Sparse Profile ($D = 8$)

```
# EXIT_1 (sparse, D=8): 1892 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented
```

```
PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_OVER 104 OP_4 OP_SUBSTR OP_CHECKLOCKTIMEVERIFY
  OP_DROP OP_OVER 72 32 OP_SUBSTR OP_ELSE OP_OVER 40 32 OP_SUBSTR OP_ENDIF
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_1 <SEL_OUT_SINGLE> OP_TX
  <0x5120> OP_3 OP_PICK OP_8 32 OP_SUBSTR OP_CAT OP_EQUALVERIFY OP_OVER OP_8
  OP_LEFT OP_FROMALTSTACK OP_IF OP_NUMEQUALVERIFY OP_ELSE OP_LESSTHANOREQUAL
  OP_VERIFY OP_ENDIF
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# JOIN_0 (sparse, D=8): 1876 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
FINAL

# JOINK_2 (sparse, D=8): 3263 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_SIZE 116 OP_EQUALVERIFY
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_PREV_S
FINAL

# UPDATE (sparse, D=8): 2092 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_2 OP_PICK 40 32 OP_SUBSTR
  OP_CHECKSIGVERIFY OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_SIZE 116
  OP_EQUALVERIFY
LEAF_POS
OP_ROT
LEAF_POS
OP_ROT OP_EQUALVERIFY OP_SWAP
LEAF_POS
SPARSE_SETUP
<TAG_NODE> OP_ROT OP_ROT
LEAF_ACC
OP_ROT
LEAF_ACC
LEVEL_S(k)  for k = 0 to 7
```

```
CHECK_OLD_S
FINAL

# CLAIM (sparse, D=8): 3386 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_DUP OP_TOALTSTACK OP_SWAP OP_TOALTSTACK
  OP_3 OP_PICK OP_3 OP_PICK OP_CAT OP_CAT
TAGGED(VOUCHER)
OP_3 OP_PICK 40 32 OP_SUBSTR OP_CHECKSIGFROMSTACK OP_VERIFY OP_SIZE 116
  OP_EQUALVERIFY OP_SWAP OP_SIZE 116 OP_EQUALVERIFY OP_SWAP OP_OVER OP_8
  OP_LEFT OP_OVER OP_8 OP_LEFT OP_SUB
LE64
OP_2 OP_PICK OP_8 104 OP_SUBSTR OP_CAT OP_2 OP_PICK 112 OP_4 OP_SUBSTR
  OP_1ADD OP_SIZE OP_4 OP_LESSTHANOREQUAL OP_VERIFY
PAD4
OP_CAT OP_FROMALTSTACK OP_ROT OP_TOALTSTACK OP_TOALTSTACK OP_OVER
LEAF_POS
OP_NIP
SPARSE_SETUP
<TAG_NODE> OP_ROT OP_ROT OP_SWAP
LEAF_ACC
OP_ROT
LEAF_ACC
OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ENDIF
LEVEL_S(k)  for k = 0 to 7
CHECK_OLD_S
OP_FROMALTSTACK OP_2 OP_ROLL OP_2 OP_ROLL OP_SWAP OP_TOALTSTACK OP_TOALTSTACK
LEAF_POS
SPARSE_SETUP
LEAF_ACC
<EMPTY_0> OP_0 OP_2SWAP
LEVEL_S(k)  for k = 0 to 7
CHECK_PREV_S
FINAL
```

C.5 FULL Numerics Without Splice or EC Opcodes ($D = 4$)

The intermediate data point of §5: BIP-441 numerics kept, while `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT` and the EC point opcodes are replaced by `OP_CAT`, `OP_SIZE` and `OP_TWEAKADD`.

```
# EXIT_1 (minimal inventory, dense, D=4): 1118 bytes
# macros in CAPITALS are defined in MACROS_FULL.txt; raw opcodes are wrapped and indented

<SEL_CURR_INPUT> OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK_MIN
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_DUP
  OP_TOALTSTACK OP_IF OP_3 OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP OP_4
  OP_PICK OP_ELSE OP_5 OP_PICK OP_ENDIF OP_CHECKSIGVERIFY OP_SIZE OP_4
  OP_EQUALVERIFY OP_OVER OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK
  OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY
  OP_DROP OP_4 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE
  32 OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_6 OP_PICK OP_TOALTSTACK OP_5 OP_PICK OP_TOALTSTACK OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT
TAGGED(LEAF)
OP_1 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_FROMALTSTACK OP_CAT OP_EQUALVERIFY
  OP_FROMALTSTACK OP_FROMALTSTACK OP_IF OP_2DUP OP_NUMEQUALVERIFY OP_ELSE
  OP_2DUP OP_LESSTHANOREQUAL OP_VERIFY OP_ENDIF OP_NIP <TAG_NODE> OP_ROT
  OP_ROT <EMPTY_0> OP_0
LEVEL_MIN  x4
CHECK_OLD
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_FROMALTSTACK
  OP_DROP OP_1 OP_ELSE OP_SWAP OP_FROMALTSTACK
```

```
TREE_SPK_MIN
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_NUMEQUAL OP_ENDIF
```

C.6 MID Tier Macros

```
# Macro definitions, MID tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(VOUCHER)
<TAG_VOUCHER> OP_SWAP OP_CAT OP_SHA256

## TAGGED(POS)
<TAG_POS> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## PADZ   ([x z w] -> [le32(x)]: z||w == 00000000 and size(x)+size(z) == 4)
OP_OVER OP_SWAP OP_CAT <ZERO4> OP_EQUALVERIFY OP_SIZE OP_ROT OP_SIZE OP_ROT
  OP_ADD OP_4 OP_NUMEQUALVERIFY OP_SWAP OP_CAT

## ENC64   ([x z w] -> [enc64(x)])
PADZ
<00000000_01> OP_CAT

## LE64   ([x z w] -> [le64(x)])
PADZ
<ZERO4> OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## PROLOGUE   (input is #0; (r, m) match the input scriptPubKey; alt <- m r)
32768 OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK_MIN
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK

## LEVEL_MID   (dense level with PADZ gadgets)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_2 OP_PICK
  OP_TOALTSTACK
ENC64
OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_7 OP_ROLL
  OP_5 OP_ROLL OP_5 OP_ROLL
ENC64
OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4 OP_PICK OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP OP_4 OP_ROLL OP_4 OP_ROLL
  OP_4 OP_ROLL
ENC64
OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_OVER
  OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
```

```
## LEVEL_MID_S   (sparse level: dir folded into the position accumulator)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_3 OP_ROLL
  OP_FROMALTSTACK OP_SWAP OP_SIZE OP_ONOTEQUAL OP_NOTIF OP_DROP OP_DUP
  OP_ENDIF OP_SWAP <ENC0> OP_CAT OP_DUP OP_CAT
TAGGED(NODE)
OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_IF OP_OVER OP_ADD OP_ENDIF
  OP_SWAP OP_DUP OP_ADD OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_3 OP_ROLL OP_3 OP_ROLL OP_3 OP_ROLL OP_2 OP_PICK OP_TOALTSTACK
ENC64
OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_7 OP_ROLL
  OP_5 OP_ROLL OP_5 OP_ROLL
ENC64
OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4 OP_PICK OP_NOTIF
  OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT OP_SHA256 OP_TOALTSTACK
  OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP OP_4 OP_ROLL OP_4 OP_ROLL
  OP_4 OP_ROLL
ENC64
OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_OVER
  OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK

## SPARSE_SETUP   ([ok nonce plo phi] -> [] ; alt <- plo 0 1 E_0)
OP_2SWAP OP_CAT
TAGGED(POS)
OP_ROT OP_DUP OP_TOALTSTACK OP_SIZE OP_4 OP_EQUALVERIFY OP_ROT OP_CAT
  OP_EQUALVERIFY OP_0 OP_TOALTSTACK OP_1 OP_TOALTSTACK <EMPTY_0>
  OP_TOALTSTACK

## SPARSE_TEARDOWN   (check the accumulated position against plo; drop bookkeeping)
OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK OP_DROP OP_FROMALTSTACK
  OP_FROMALTSTACK OP_NUMEQUALVERIFY

## CHECK_OLD   (os (padded) == input 0 amount, oh == r)
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_OLD_S   (CHECK_OLD with the sparse teardown)
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
<SEL_IN_AMOUNT> OP_TX OP_EQUALVERIFY
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV   (os == ns_prev, oh == nh_prev; chained steps)
OP_2SWAP OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## CHECK_PREV_S   (CHECK_PREV with the sparse teardown)
OP_2SWAP
SPARSE_TEARDOWN
OP_FROMALTSTACK OP_NUMEQUALVERIFY OP_FROMALTSTACK OP_EQUALVERIFY

## LEAF_JOIN   ([amt8 ek ok evk ea nonce seq amt_num za wa] -> [leaf] | amt_num)
OP_2 OP_PICK OP_TOALTSTACK
LE64
OP_7 OP_PICK OP_EQUALVERIFY OP_DUP OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_1
  OP_PICK OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK OP_SIZE OP_4
  OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_4
  OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE 32
  OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT

## FINAL   [shown for D = 4; <EMPTY_D> is the empty root of the tree's depth]   (output 0 is
     (le64(ns), TREE_SPK(nh, m)) unless nh == E_D)
OP_ROT OP_DROP OP_OVER <EMPTY_4> OP_EQUAL OP_IF OP_2DROP OP_2DROP
  OP_FROMALTSTACK OP_DROP OP_1 OP_ELSE OP_3 OP_ROLL OP_3 OP_ROLL
LE64
```

```
OP_SWAP OP_FROMALTSTACK
TREE_SPK_MIN
OP_0 <SEL_OUT_SINGLE> OP_TX OP_ROT OP_EQUALVERIFY OP_EQUAL OP_ENDIF
```

C.7 MID Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (MID tier, dense, D=4): 1427 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
OP_DUP OP_TOALTSTACK OP_IF OP_3 OP_PICK OP_CHECKLOCKTIMEVERIFY OP_DROP OP_10
  OP_PICK OP_CHECKSIGVERIFY
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_ELSE OP_11 OP_PICK OP_CHECKSIGVERIFY OP_2DROP
  OP_DROP OP_ENDIF OP_1 <SEL_OUT_SINGLE> OP_TX <0x5120> OP_11 OP_PICK OP_CAT
  OP_EQUALVERIFY OP_10 OP_PICK OP_EQUALVERIFY OP_FROMALTSTACK OP_DROP
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_MID  x4
CHECK_OLD
FINAL

# JOIN_0 (MID tier, dense, D=4): 1264 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP
  <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_OLD
FINAL

# JOIN_2 (MID tier, dense, D=4): 952 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP
  <TAG_NODE> OP_ROT OP_ROT <EMPTY_2> OP_0 OP_2SWAP
LEVEL_MID  x2
CHECK_OLD
FINAL

# JOINK_2 (MID tier, dense, D=4): 2005 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY OP_OVER OP_SIZE 32 OP_EQUALVERIFY OP_DROP
  <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_OLD
OP_SWAP OP_TOALTSTACK OP_TOALTSTACK OP_DROP OP_OVER OP_SIZE 32 OP_EQUALVERIFY
  OP_DROP <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
FINAL

# UPDATE (MID tier, dense, D=4): 1535 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
<OPERATOR_KEY> OP_CHECKSIGVERIFY 18 OP_PICK OP_CHECKSIGVERIFY
LEAF_JOIN
OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
```

```
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_SWAP
TAGGED(LEAF)
OP_SWAP <TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
FINAL

# UPDATE_X (MID tier, exit-key consent, dense, D=4): 1539 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

PROLOGUE
21 OP_PICK OP_CHECKSIGVERIFY <OPERATOR_KEY> OP_CHECKSIGVERIFY 18 OP_PICK
  OP_CHECKSIGVERIFY
LEAF_JOIN
OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_SWAP
TAGGED(LEAF)
OP_SWAP <TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
FINAL

# CLAIM (MID tier, dense, D=4): 2547 bytes
# macros in CAPITALS are defined in MACROS_MID.txt; raw opcodes are wrapped and indented

32768 OP_TX OP_0 OP_NUMEQUALVERIFY OP_2DUP
TREE_SPK_MIN
<SEL_IN_SPK> OP_TX OP_EQUALVERIFY OP_DUP OP_TOALTSTACK OP_SWAP OP_TOALTSTACK
  28 OP_PICK 28 OP_PICK 28 OP_PICK 28 OP_PICK 28 OP_PICK 28 OP_PICK 28
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT 19 OP_PICK 19 OP_PICK 19
  OP_PICK 19 OP_PICK 19 OP_PICK 19 OP_PICK 19 OP_PICK OP_CAT OP_CAT OP_CAT
  OP_CAT OP_CAT OP_CAT OP_2 OP_PICK OP_ROT OP_ROT OP_CAT OP_CAT
TAGGED(VOUCHER)
OP_ROT OP_SWAP 27 OP_PICK OP_CHECKSIGFROMSTACK OP_VERIFY OP_DROP 20 OP_PICK
  OP_7 OP_PICK OP_7 OP_ROLL OP_7 OP_ROLL
PADZ
OP_EQUALVERIFY OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK
  OP_TOALTSTACK
LEAF_JOIN
OP_TOALTSTACK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8 OP_PICK OP_8
  OP_PICK OP_CAT OP_CAT OP_CAT OP_CAT OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK OP_DUP OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK OP_7 OP_PICK OP_TOALTSTACK OP_6 OP_PICK
  OP_TOALTSTACK OP_TOALTSTACK OP_6 OP_ROLL OP_DROP OP_7 OP_ROLL OP_6 OP_ROLL
  OP_SUB OP_DUP OP_0 OP_GREATERTHANOREQUAL OP_VERIFY OP_DUP OP_3 OP_ROLL OP_3
  OP_ROLL
LE64
OP_4 OP_ROLL OP_1ADD OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
OP_3 OP_ROLL OP_SWAP OP_CAT OP_CAT
TAGGED(LEAF)
OP_SWAP OP_DUP OP_NOT OP_IF OP_SWAP OP_DROP <EMPTY_0> OP_SWAP OP_ENDIF
  <TAG_NODE> OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
LEVEL_MID  x4
CHECK_OLD
OP_FROMALTSTACK OP_FROMALTSTACK
TAGGED(LEAF)
OP_SWAP OP_2SWAP OP_SWAP OP_TOALTSTACK OP_TOALTSTACK <EMPTY_0> OP_0 OP_2SWAP
LEVEL_MID  x4
CHECK_PREV
FINAL
```

C.8 MIN Tier Macros

```
# Macro definitions, MIN tier. Each expansion is itself collapsed
# against the other macros; a macro never contains itself.

## TAGGED(LEAF)
<TAG_LEAF> OP_SWAP OP_CAT OP_SHA256

## TAGGED(NODE)
<TAG_NODE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(STATE)
<TAG_STATE> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPTWEAK)
<TAG_TAPTWEAK> OP_SWAP OP_CAT OP_SHA256

## TAGGED(TAPSIGHASH)
<TAG_TAPSIGHASH> OP_SWAP OP_CAT OP_SHA256

## TAGGED(CHALLENGE)
<TAG_CHALLENGE> OP_SWAP OP_CAT OP_SHA256

## PADZ   ([x z w] -> [le32(x)]: z||w == 00000000 and size(x)+size(z) == 4)
OP_OVER OP_SWAP OP_CAT <ZERO4> OP_EQUALVERIFY OP_SIZE OP_ROT OP_SIZE OP_ROT
  OP_ADD OP_4 OP_NUMEQUALVERIFY OP_SWAP OP_CAT

## LE64   ([x z w] -> [le64(x)])
PADZ
<ZERO4> OP_CAT

## TREE_SPK_MIN   ([r m] -> [TREE_SPK(r, m)] with OP_TWEAKADD)
OP_SWAP
TAGGED(STATE)
<NUMS_X> OP_TWEAKADD OP_TUCK OP_SWAP OP_CAT
TAGGED(TAPTWEAK)
OP_SWAP OP_TWEAKADD <0x5120> OP_SWAP OP_CAT

## LEVEL_MID   (dense level with PADZ gadgets)
OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13
  OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_13 OP_ROLL OP_2 OP_PICK
  OP_TOALTSTACK
PADZ
<00000000_01> OP_CAT OP_CAT OP_FROMALTSTACK OP_7 OP_PICK OP_OVER OP_ADD
  OP_TOALTSTACK OP_7 OP_ROLL OP_5 OP_ROLL OP_5 OP_ROLL
PADZ
<00000000_01> OP_CAT OP_6 OP_ROLL OP_SWAP OP_CAT OP_2 OP_PICK OP_SWAP OP_4
  OP_PICK OP_NOTIF OP_SWAP OP_ENDIF OP_CAT OP_8 OP_PICK OP_SWAP OP_CAT
  OP_SHA256 OP_TOALTSTACK OP_5 OP_PICK OP_OVER OP_ADD OP_TOALTSTACK OP_DROP
  OP_4 OP_ROLL OP_4 OP_ROLL OP_4 OP_ROLL
PADZ
<00000000_01> OP_CAT OP_3 OP_ROLL OP_SWAP OP_CAT OP_ROT OP_NOTIF OP_SWAP
  OP_ENDIF OP_CAT OP_OVER OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK
  OP_FROMALTSTACK OP_FROMALTSTACK

## LEAF_JOIN   ([amt8 ek ok evk ea nonce seq amt_num za wa] -> [leaf] | amt_num)
OP_2 OP_PICK OP_TOALTSTACK
LE64
OP_7 OP_PICK OP_EQUALVERIFY OP_DUP OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_1
  OP_PICK OP_SIZE OP_4 OP_EQUALVERIFY OP_DROP OP_2 OP_PICK OP_SIZE OP_4
  OP_EQUALVERIFY OP_DROP OP_3 OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_4
  OP_PICK OP_SIZE 32 OP_EQUALVERIFY OP_DROP OP_5 OP_PICK OP_SIZE 32
  OP_EQUALVERIFY OP_DROP OP_6 OP_PICK OP_SIZE OP_8 OP_EQUALVERIFY OP_DROP
  OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT OP_CAT
```

C.9 MIN Tier, Dense Profile ($D = 4$)

```
# EXIT_1 (MIN tier, dense, D=4): 1700 bytes
# macros in CAPITALS are defined in MACROS_MIN.txt; raw opcodes are wrapped and indented

OP_2DUP
TREE_SPK_MIN
OP_TOALTSTACK OP_TOALTSTACK OP_TOALTSTACK OP_IF OP_3 OP_PICK
  OP_CHECKLOCKTIMEVERIFY OP_DROP OP_10 OP_PICK OP_CHECKSIGVERIFY
PADZ
OP_6 OP_PICK OP_EQUALVERIFY OP_ELSE OP_11 OP_PICK OP_CHECKSIGVERIFY OP_2DROP
  OP_DROP OP_ENDIF OP_9 OP_PICK 2117922 OP_CAT OP_9 OP_PICK OP_CAT
  OP_TOALTSTACK
LEAF_JOIN
TAGGED(LEAF)
OP_FROMALTSTACK <TAG_NODE> OP_ROT OP_ROT <EMPTY_0> OP_0
LEVEL_MID  x4
OP_2SWAP OP_6 OP_ROLL OP_6 OP_ROLL
LE64
OP_FROMALTSTACK OP_FROMALTSTACK OP_3 OP_ROLL OP_EQUALVERIFY OP_4 OP_ROLL
  OP_DROP OP_ROT OP_5 OP_ROLL OP_5 OP_ROLL
LE64
OP_3 OP_ROLL OP_FROMALTSTACK
TREE_SPK_MIN
34 OP_SWAP OP_CAT OP_CAT OP_SWAP OP_CAT OP_2 OP_ROLL OP_CAT OP_SHA256 OP_2
  OP_ROLL OP_ROT OP_SWAP OP_CAT OP_SHA256 OP_FROMALTSTACK 34 OP_SWAP OP_CAT
  OP_3 OP_ROLL OP_CAT OP_SHA256 OP_3 OP_ROLL OP_4 OP_ROLL 0 OP_SWAP OP_CAT
  OP_SWAP OP_CAT OP_4 OP_ROLL OP_CAT OP_ROT OP_CAT OP_SWAP OP_CAT OP_3
  OP_ROLL OP_CAT OP_SWAP OP_CAT <0200000000> OP_CAT OP_SWAP OP_CAT
  <00ffffffff> OP_CAT
TAGGED(TAPSIGHASH)
<G_X||G_X> OP_SWAP OP_CAT
TAGGED(CHALLENGE)
OP_OVER 0 OP_CAT OP_EQUALVERIFY <G_X> OP_SWAP OP_CAT OP_1 OP_CAT <G_X>
  OP_CHECKSIG
```